

Projet S8 - QML

Eliott Saltre,
Clément Perrin,
Esteban Carlin,
Kris Joubi

Mai 2024

Table des matières

I	Comparaison de certains algorithmes Quantiques et Classiques sur un ordinateur classique	3
1	Définitions et outils	3
1.1	Machine Learning Classique (ML ou CML)	3
1.2	Machine Learning Quantique (QML)	3
1.3	Qubits et Algorithme de Grover	3
1.4	Critères de comparaison du fonctionnement et des performances	3
2	SVM Quantique vs Classique	4
2.1	Présentation conceptuelle	4
2.2	Fonctionnement Classique	4
2.2.1	Principe de base des SVM	4
2.2.2	Fonctionnement des SVM (et SVC)	4
2.2.3	Support Vector Classifier (SVC)	5
2.2.4	Points clés des SVM/SVC	5
2.3	Fonctionnement Quantique	5
2.3.1	Concept de base du Quantum Support Vector Classifier (QSVC)	6
2.3.2	Étapes du QSVC	6
2.3.3	Avantages potentiels du QSVC	6
2.3.4	Points clés de l'implémentation	7
2.3.5	Limitations actuelles	7
2.4	Comparaisons	7
2.4.1	Iris	7
3	Perceptron Quantique vs Classique	10
3.1	Présentation conceptuelle	10
3.2	Fonctionnement Classique	10
3.2.1	Fonctionnement du Perceptron	10
3.2.2	Exemple Illustratif	10
3.3	Fonctionnement Quantique	11
3.3.1	Concepts de Base	11
3.3.2	Fonctionnement du Perceptron Quantique	11
3.4	Comparaisons	14
II	Algorithmes quantiques sur ordinateur quantique	16

4	Quelques prérequis d'informatique quantique	16
4.1	Principe de base du qubit	16
4.2	Conditions nécessaires au fonctionnement d'un ordinateur quantique	16
4.3	Principales portes des ordinateurs quantiques	17
4.3.1	Portes de Pauli X, Y et Z	17
4.3.2	Porte de Hadamard	17
4.3.3	Porte C-NOT	17
4.3.4	Autres portes	18
4.4	L'état Greenberger–Horne–Zeilinger (GHZ)	18
5	Introduction au calcul quantique	18
5.1	Introduction aux différents modules de <i>qiskit</i>	18
5.1.1	Présentation des modules	18
5.1.2	Principes et exemple d'utilisation	19
5.2	Utilisation d'ordinateurs quantiques	20
5.2.1	Intérêt et mise en place de l'environnement	20
5.2.2	Transpilers	21
5.2.3	Exemple minimal	21
5.2.4	Exemple mettant en lumière le bruit des processeurs quantiques	24
6	Informations sur les ordinateurs quantiques utilisés	26
6.1	Principaux processeurs actuels d'IBM	26
6.2	<i>Error Per Layered Gate</i> (EPLG)	26
6.3	Caractéristiques des ordinateurs utilisés	27
7	Exploration d'algorithmes quantiques	27
7.1	Algorithme de Grover	27
7.1.1	Principe	27
7.1.2	Implémentation du circuit quantique	27
7.1.3	Optimisation, exécution et résultats	32
7.2	Classification de type k-means	33
7.2.1	Problème du k-means quantique	33
7.2.2	Distance Euclidienne calculée par ordinateur quantique	34

Première partie

Comparaison de certains algorithmes Quantiques et Classiques sur un ordinateur classique

1 Définitions et outils

L'apprentissage automatique est un sous-ensemble de l'intelligence artificielle (IA) qui permet aux systèmes informatiques d'apprendre par l'expérience sans être spécifiquement programmés. Les algorithmes d'apprentissage automatique sont basés sur un modèle mathématique qui extrait des informations des données existantes pour créer un modèle prédictif. Jusqu'à récemment, les ordinateurs classiques étaient la seule option disponible pour la formation et l'exécution des modèles de ML. Cependant, avec l'avènement de l'informatique quantique (QC), les chercheurs ont commencé à explorer les avantages potentiels des algorithmes quantiques pour les applications d'apprentissage automatique. [SSP14]

1.1 Machine Learning Classique (ML ou CML)

Les algorithmes CML sont basés sur l'informatique classique, qui utilise des bits ayant une valeur de 1 ou 0 pour représenter les données. Malgré les limites de l'informatique classique, les algorithmes CML ont fait preuve d'un succès remarquable dans la résolution d'un large éventail de tâches, allant de la reconnaissance d'images et de la parole à la traduction linguistique et à la détection des fraudes. Avec la croissance exponentielle de la production de données et le besoin croissant de les traiter et de les analyser efficacement, le CML est devenu une technologie essentielle dans de nombreux domaines, notamment la finance, les soins de santé et le commerce de détail. [Sol64] En outre, les algorithmes CML sont généralement plus faciles à mettre en œuvre et à interpréter que les algorithmes QML, ce qui en fait un choix privilégié pour de nombreux développeurs et entreprises.

1.2 Machine Learning Quantique (QML)

L'informatique quantique repose sur les principes de la mécanique quantique, qui permet une puissance de calcul plus importante que l'informatique classique. Les algorithmes QML utilisent des qubits, qui peuvent avoir une valeur de 0, 1 ou les deux simultanément, pour représenter des structures de données complexes. Le QML a le potentiel de surpasser les algorithmes CML pour certaines applications, principalement en raison de son importante puissance de calcul. Par exemple, QML peut être utilisé pour résoudre des problèmes d'optimisation, tels que l'optimisation de portefeuille (finance), plus efficacement que CML.

Dans ce projet nous essayons de comparer l'apprentissage automatique quantique (QML) et l'apprentissage automatique classique (CML) afin de déterminer les avantages, les inconvénients et les perspectives que chacun propose. [Sim22]

1.3 Qubits et Algorithme de Grover

Le qubit est un système quantique à deux niveaux, tandis que l'algorithme de Grover est un algorithme quantique sur ordinateur quantique ayant la particularité d'avoir une complexité en $O(\sqrt{N})$ pour une liste de taille N . Nous définirons ces deux notions plus en détails dans le paragraphe 4.1 pour le qubit et le paragraphe 7.1, où nous revenons en détail sur l'algorithme de Grover.

1.4 Critères de comparaison du fonctionnement et des performances

Les critères utilisés sont arbitraires. Nous nous assurons de la reproductibilité et de la fiabilité des comparaisons en utilisant les mêmes données, les mêmes seeds et en réalisant des cross-validations assurant une évaluation consistante. Nous tâcherons de prendre en compte le temps d'exécution à performances égales pour en faire une mesure supplémentaire.

Nous regarderons lequel des algorithmes est le plus efficace et performant. Les données utilisées seront tirées de bibliothèques python tels que Keras et Scikit-Learn, en particulier :

— **Iris (Scikit-learn)** :

Il s'agit peut-être de la base de données la plus connue dans la littérature sur la reconnaissance des formes. L'article de Fisher est un classique dans le domaine et est fréquemment cité en référence à ce jour (voir [DHG01] par exemple). L'ensemble de données contient 3 classes de 50 instances chacune, où chaque classe se réfère à un type de plante d'iris. Une classe est linéairement séparable des deux autres ; ces dernières ne sont PAS linéairement séparables les unes des autres. [Kan+18]

2 SVM Quantique vs Classique

2.1 Présentation conceptuelle

L'un des algorithmes QML les plus prometteurs est la Quantum Support Vector Machine (QSVM). La QSVM a montré des performances supérieures à celles de la SVM classique pour les problèmes de classification. Plus précisément, une étude récente menée par IBM a montré que, pour certains ensembles de données, l'algorithme QSVM était jusqu'à 100 fois plus rapide que l'algorithme SVM classique.

Dans ce rapport, nous nous concentrerons sur les machines à vecteurs de support, qui sont conçues pour les problèmes de classification sur de petits ensembles de données et qui ont un équivalent QSVM quantique. (QSVC). [OK23]

2.2 Fonctionnement Classique

Les **Support Vector Machines** (SVM) sont des algorithmes d'apprentissage supervisé utilisés principalement pour des tâches de classification et de régression. Les SVM sont particulièrement puissants pour les problèmes de classification linéaire et non linéaire. Une variante spécifique des SVM pour la classification est le **Support Vector Classifier** (SVC). [CV95]

2.2.1 Principe de base des SVM

Les SVM cherchent à trouver l'hyperplan optimal qui sépare les classes de données dans l'espace des caractéristiques. Un hyperplan est une frontière de décision linéaire qui divise l'espace en deux parties, chaque partie correspondant à une classe différente. L'objectif est de maximiser la marge entre les données des deux classes, où la marge est définie comme la distance entre l'hyperplan et les points de données les plus proches de chaque classe, appelés vecteurs de support.

2.2.2 Fonctionnement des SVM (et SVC)

Séparation Linéaire - Pour des données linéairement séparables, les SVM cherchent l'hyperplan qui maximise la marge entre les classes.

- Mathématiquement, cela peut être formulé comme un problème d'optimisation quadratique :

$$\text{minimiser } \frac{1}{2} \|w\|^2$$

$$\text{sous les contraintes } y_i(w \cdot x_i + b) \geq 1 \quad \text{pour tout } i$$

où w est le vecteur de poids, b est le biais, x_i est un vecteur de caractéristiques, et y_i est la classe associée (+1 ou -1).

Non-linéarité - Pour les données non-linéairement séparables, les SVM utilisent l'"astuce du noyau (kernel trick)" pour transformer les données dans un espace de plus haute dimension où une séparation linéaire est possible.

- Le problème d'optimisation devient alors :

$$\text{minimiser } \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

$$\text{sous les contraintes } y_i(w \cdot \phi(x_i) + b) \geq 1 - \xi_i \quad \text{et} \quad \xi_i \geq 0$$

où ξ_i sont des variables slack permettant de tolérer les erreurs de classification et C est un paramètre de régularisation qui contrôle le compromis entre maximisation de la marge et minimisation de l'erreur de classification.

2.2.3 Support Vector Classifier (SVC)

Le Support Vector Classifier (SVC) est l'implémentation spécifique des SVM pour les tâches de classification. Dans la bibliothèque Python 'scikit-learn', l'algorithme SVC est disponible et permet de créer des modèles de classification basés sur SVM. [Bur98]

Exemple d'implémentation en Python avec 'scikit-learn' :

```
1 from sklearn import datasets
2 from sklearn.model_selection import train_test_split
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.svm import SVC
5 from sklearn.metrics import accuracy_score
6
7 # Chargement du dataset
8 iris = datasets.load_iris()
9 X = iris.data
10 y = iris.target
11
12 # Séparation des données en jeu d'entraînement et de test
13 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
14                                                    random_state=42)
15
16 # Normalisation des données
17 scaler = StandardScaler()
18 X_train = scaler.fit_transform(X_train)
19 X_test = scaler.transform(X_test)
20
21 # Création et entraînement du modèle SVC
22 svc = SVC(kernel='rbf', C=1.0, gamma='scale')
23 svc.fit(X_train, y_train)
24
25 # Prédiction
26 y_pred = svc.predict(X_test)
27
28 # Évaluation du modèle
29 accuracy = accuracy_score(y_test, y_pred)
30 print(f"Accuracy: {accuracy}")
```

2.2.4 Points clés des SVM/SVC

- Hyperplans et Marges : Les SVM cherchent l'hyperplan avec la marge maximale pour séparer les classes.
- Vecteurs de Support : Les points de données les plus proches de l'hyperplan, influençant sa position et orientation.
- Noyaux : Utilisés pour transformer les données non linéaires dans un espace de caractéristiques de plus haute dimension où une séparation linéaire est possible.
- Paramètre C : Contrôle le compromis entre une marge large et la classification correcte des points de données.
- Complexité : Les SVM peuvent être coûteux en termes de calcul et de mémoire, surtout pour les grands ensembles de données.

2.3 Fonctionnement Quantique

L'équivalent quantique de l'algorithme SVM/SVC exploite les capacités de calcul quantique pour traiter des données de manière plus efficace, notamment grâce à la superposition et l'intrication quantiques. Un exemple de cet équivalent est le Quantum Support Vector Classifier (QSVC), implémenté dans la bibliothèque Qiskit de IBM [Sha+22].

2.3.1 Concept de base du Quantum Support Vector Classifier (QSVC)

Le QSVC fonctionne sur le même principe de base que les SVM classiques : il cherche à trouver une frontière de décision optimale entre les classes de données. Cependant, il utilise des circuits quantiques pour effectuer certaines étapes de calcul, ce qui peut potentiellement offrir des avantages en termes de vitesse et de capacité de traitement pour certains types de données et problèmes. [SHM24], [Par+20].

2.3.2 Étapes du QSVC

Encodage des données Les données classiques sont encodées dans des états quantiques. Cela se fait généralement à l'aide de circuits quantiques qui préparent un état quantique en fonction des valeurs des caractéristiques des données.

Par exemple, une méthode courante est l'encodage angulaire où chaque caractéristique est mappée sur un angle de rotation d'un qubit.

Calcul du noyau quantique L'étape clé qui différencie QSVC des SVM classiques est l'utilisation d'un noyau quantique.

- Un noyau quantique est défini par l'état de superposition des qubits et permet de calculer la similarité entre les points de données dans l'espace des caractéristiques quantiques.
- La fonction de noyau quantique est souvent calculée en utilisant la "fidélité quantique", qui mesure la similitude entre deux états quantiques.

Optimisation de l'hyperplan Comme dans les SVM classiques, QSVC cherche à maximiser la marge entre les classes de données. Cependant, ici, les calculs de similarité (ou de noyau) sont réalisés via des circuits quantiques.

Une fois que les noyaux sont calculés, une optimisation classique similaire à celle des SVM est utilisée pour trouver l'hyperplan optimal dans l'espace des caractéristiques quantiques.

2.3.3 Avantages potentiels du QSVC

- Efficacité dans l'espace des caractéristiques : Les circuits quantiques peuvent potentiellement explorer un espace de caractéristiques beaucoup plus large et complexe, ce qui peut aider à trouver des séparations plus efficaces entre les classes de données.
- Calculs parallèles : Les ordinateurs quantiques peuvent effectuer certains calculs en parallèle grâce à la superposition, ce qui peut réduire le temps de calcul pour des tâches spécifiques.
- Résolution de problèmes complexes : Pour certains problèmes très complexes ou à haute dimensionnalité, les algorithmes quantiques peuvent offrir des avantages en termes de vitesse de convergence et de performance par rapport aux algorithmes classiques.

[OK23]

Exemple d'implémentation avec Qiskit Qiskit fournit des outils pour implémenter des QSVC, notamment via le module 'qiskit-machine-learning'. Voici un exemple d'implémentation :

```
1 from qiskit import BasicAer
2 from qiskit.utils import QuantumInstance
3 from qiskit_machine_learning.algorithms import QSVC
4 from qiskit_machine_learning.kernels import QuantumKernel
5 from qiskit.circuit.library import ZZFeatureMap
6 from sklearn.datasets import load_iris
7 from sklearn.model_selection import train_test_split
8 from sklearn.preprocessing import StandardScaler
9
10 # Chargement du dataset Iris
11 data = load_iris()
12 X = data.data
13 y = data.target
14
15 # Filtrage pour simplifier le problème de classification binaire
16 X = X[y != 2]
17 y = y[y != 2]
18
19 # Prétraitement des données
20 scaler = StandardScaler()
21 X = scaler.fit_transform(X)
```

```

22
23 # Séparation des données en jeu d'entraînement et de test
24 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)
25
26 # Définition de la carte de caractéristiques quantiques (feature map)
27 feature_map = ZZFeatureMap(feature_dimension=4, reps=2, entanglement='linear')
28
29 # Définition du noyau quantique
30 quantum_kernel = QuantumKernel(feature_map=feature_map, quantum_instance=
    QuantumInstance(BasicAer.get_backend('qasm_simulator')))
31
32 # Création et entraînement du modèle QSVC
33 qsvc = QSVC(quantum_kernel=quantum_kernel)
34 qsvc.fit(X_train, y_train)
35
36 # Prédiction et évaluation
37 y_pred = qsvc.predict(X_test)
38 accuracy = (y_test == y_pred).mean()
39 print(f"Accuracy: {accuracy}")

```

2.3.4 Points clés de l'implémentation

- Encodage des caractéristiques : Utilisation de 'ZZFeatureMap' pour encoder les caractéristiques des données dans des états quantiques.
- Noyau quantique : 'QuantumKernel' utilise la fonction de noyau quantique pour mesurer la similarité entre les données.
- Simulation quantique : Utilisation de 'BasicAer.get_backend('qasm_simulator')' pour simuler le calcul quantique. Dans une implémentation réelle, cela serait remplacé par un backend d'ordinateur quantique réel.

2.3.5 Limitations actuelles

- Bruiteur et décohérence : Les ordinateurs quantiques actuels sont encore limités par le bruit et la décohérence, ce qui peut affecter la précision des calculs.
- Ressources et scalabilité : Les ordinateurs quantiques disponibles aujourd'hui ont un nombre limité de qubits et de ressources, ce qui peut limiter leur application à des problèmes de grande envergure.

2.4 Comparaisons

La comparaison a été réalisée sur le dataset Iris.

2.4.1 Iris

Les colonnes de cet ensemble de données sont les suivantes :

- Id unique
- Longueur du sépale (cm)
- Largeur du sépale (cm)
- Longueur du pétale (cm)
- Largeur du pétale (cm)
- Espèces (label ou réponse) [[Kan+18](#)]

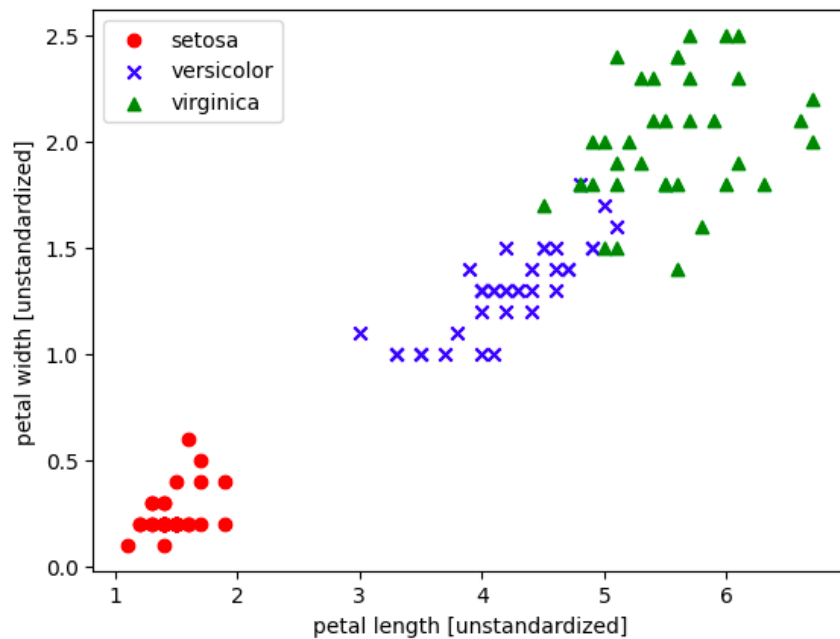


FIGURE 1 – Visualisation des données avant standardisation

Ci-dessus nous avons une visualisation de la base de données Iris, elle comporte bien 3 classes (correspondant aux 3 espèces de fleurs), où l'une d'entre elles est linéairement séparable des autres mais pas les autres.

On fit le SVC et on observe le résultat :

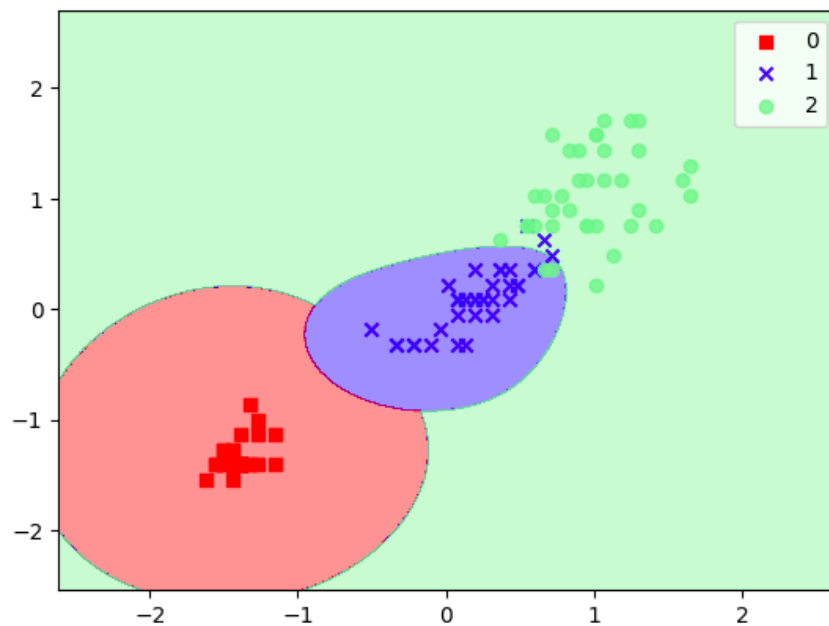


FIGURE 2 – Régions de classification

Après avoir fait tourner l'algorithme et avoir tracé les zones/frontières de décision on obtient des résultats visuellement bons. Les 3 catégories sont clairement délimitées et hormis quelques points qui ont l'air de faire exception, les zones semblent correctes.. Néanmoins, ce n'est pas un argument suffisant et ce sont les résultats des données d'entraînement. Nous allons calculer la précision (nombre de succès sur nombre total de cas) pour en avoir le coeur net.


```

Predicted: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
0 0 0 2 1 1 0 0]
Which was [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
0 0 0 2 1 1 0 0]
Overall accuracy of: 1.0
Overall accuracy (with train points) of: 0.9523809523809523

```

FIGURE 3 – Résultats de test

Ici le score est de 100% de précision, la tâche étant aisée et classique, c'était attendu. L'entraînement s'est légèrement moins bien passé que le test mais la différence est faible et le hasard en est probablement la cause (la distribution des jeux de données ou des classes par exemple). En revanche, penchons-nous sur les résultats de notre algorithme de QSVC :

```

Predicted: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 1 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
0 0 0 2 1 1 0 0]
Which was [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
0 0 0 2 1 1 0 0]
Overall accuracy of: 1.0
Overall accuracy (with train points) of: 1.0

```

FIGURE 4 – Résultats de l'algorithme quantique

Tout d'abord la précision est aussi de 1. Intéressant, les performances semblent suivre. En revanche lorsque l'on se penche sur les régions de décision, on observe quelque chose de beaucoup plus complexe que dans le cas classique :

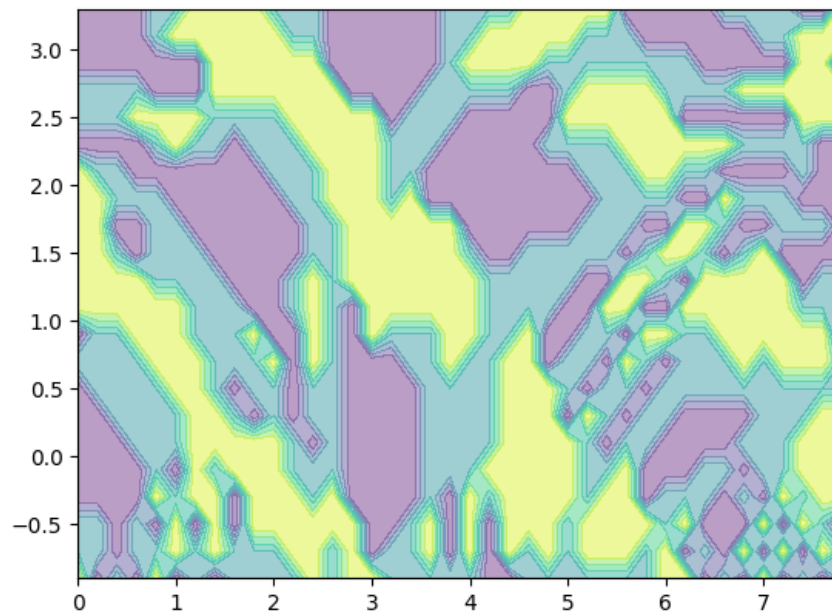


FIGURE 5 – Frontières de décision

En effet, l'aspect quantique de l'algorithme permet des frontières très complexes relativement facilement, mais dans notre cas ce n'est pas nécessaire. Le temps d'exécution était également plus long, environ 2 fois plus long d'après nos observations et mesures. 2 fois 5 secondes reste un temps léger mais significatif si les ordres de grandeur augmentent.

Il reste à savoir si un effet d'échelle existe ou non. A ce stade nous n'avons pas encore approfondi la comparaison avec d'autres bases de données, ce serait la prochaine étape.

3 Perceptron Quantique vs Classique

3.1 Présentation conceptuelle

Dans le domaine du Machine Learning, le perceptron (ou neurone de McCulloch-Pitts) est un algorithme d'apprentissage supervisé de classificateurs binaires. Un classificateur binaire est une fonction capable de décider si une entrée, représentée par un vecteur de nombres, appartient ou non à une classe spécifique. Il s'agit d'un type de classificateur linéaire, c'est-à-dire d'un algorithme de classification qui effectue ses prédictions sur la base d'une fonction prédictive linéaire combinant un ensemble de poids avec le vecteur de caractéristiques.

3.2 Fonctionnement Classique

Le perceptron est un algorithme de classification binaire utilisé dans l'apprentissage supervisé. Développé par Frank Rosenblatt [Ros58] dans les années 1950, c'est l'un des algorithmes les plus simples et constitue la base des réseaux de neurones plus complexes. Son antécédent fut développé par McCulloch et Pitts et s'inspirait fortement des neurones cérébrales. [MP43]

3.2.1 Fonctionnement du Perceptron

Architecture - Entrées : $x_1, x_2, \dots, x_n$ - Poids : $w_1, w_2, \dots, w_n$ - Biais : b - Sortie : y (soit 0 soit 1 pour un perceptron classique)

Calcul de la sortie

- Somme pondérée : Le perceptron calcule une somme pondérée de ses entrées. Cette somme est donnée par :

$$z = \sum_{i=1}^n w_i x_i + b$$

- Fonction d'activation : Pour déterminer la sortie finale, cette somme pondérée passe par une fonction d'activation. Dans le cas d'un perceptron simple, on utilise souvent la fonction de Heaviside ou la fonction de seuil :

$$y = \begin{cases} 1 & \text{si } z \geq 0 \\ 0 & \text{sinon} \end{cases}$$

Apprentissage

- Initialisation : Les poids w_i et le biais b sont initialisés (souvent à des petites valeurs aléatoires).
- Mise à jour des poids : L'algorithme apprend en ajustant les poids et le biais en fonction des erreurs commises sur un ensemble d'entraînement. Pour chaque exemple de formation (x, label) :
 - Calculer la prédiction $\hat{y}$ en utilisant les poids et le biais actuels.
 - Calculer l'erreur $\text{erreur} = \text{label} - \hat{y}$.
 - Mettre à jour les poids et le biais en utilisant la règle de mise à jour suivante :

$$w_i \leftarrow w_i + \eta \cdot \text{erreur} \cdot x_i$$

$$b \leftarrow b + \eta \cdot \text{erreur}$$

où η est le taux d'apprentissage.

3.2.2 Exemple Illustratif

Supposons que nous avons un jeu de données simple pour la classification de points dans un plan en deux dimensions.

- Données :
 - $(x_1, x_2) = (2, 3)$ avec label 1
 - $(x_1, x_2) = (1, 1)$ avec label 0

- Initialisation :
 - Poids $w_1 = 0.5$, $w_2 = -0.5$
 - Biais $b = 0.0$
 - Taux d'apprentissage $\eta = 0.1$
- Iteration 1 :
 1. Pour (2, 3) :
 - Somme pondérée : $z = 0.5 \cdot 2 + (-0.5) \cdot 3 + 0.0 = 1 - 1.5 = -0.5$
 - Prédiction : $\hat{y} = 0$
 - Erreur : $1 - 0 = 1$
 - Mise à jour des poids :

$$w_1 \leftarrow 0.5 + 0.1 \cdot 1 \cdot 2 = 0.7$$

$$w_2 \leftarrow -0.5 + 0.1 \cdot 1 \cdot 3 = -0.2$$

$$b \leftarrow 0.0 + 0.1 \cdot 1 = 0.1$$

2. Pour (1, 1) :
 - Somme pondérée : $z = 0.7 \cdot 1 + (-0.2) \cdot 1 + 0.1 = 0.6$
 - Prédiction : $\hat{y} = 1$
 - Erreur : $0 - 1 = -1$
 - Mise à jour des poids :

$$w_1 \leftarrow 0.7 + 0.1 \cdot (-1) \cdot 1 = 0.6$$

$$w_2 \leftarrow -0.2 + 0.1 \cdot (-1) \cdot 1 = -0.3$$

$$b \leftarrow 0.1 + 0.1 \cdot (-1) = 0.0$$

3.3 Fonctionnement Quantique

Le perceptron quantique est une extension du perceptron classique dans le domaine de l'informatique quantique. Il combine les principes de l'apprentissage automatique avec les avantages de la computation quantique pour potentiellement offrir des gains de performance significatifs. [WKS16]

3.3.1 Concepts de Base

Superposition et Interférence

- Superposition : Les qubits (bits quantiques) peuvent exister dans un état de superposition, représentant simultanément les états 0 et 1. Cette propriété permet d'explorer plusieurs solutions en parallèle.
- Interférence : Les états quantiques peuvent interférer entre eux, ce qui peut amplifier les probabilités des solutions correctes et annuler les incorrectes.

Algorithme de Grover : Utilisé pour la recherche dans une base de données non structurée, l'algorithme de Grover offre une accélération quadratique par rapport aux algorithmes classiques. Dans le contexte des perceptrons quantiques, il peut être utilisé pour optimiser les poids en explorant simultanément plusieurs configurations. [Gro96a] 7.1

3.3.2 Fonctionnement du Perceptron Quantique

Fonctionnement du Perceptron Quantique en Détail Le perceptron quantique combine les principes du perceptron classique avec les avantages de la mécanique quantique, tels que la superposition, l'intrication et l'interférence. Voici une présentation plus détaillée de son fonctionnement, en se concentrant sur chaque étape clé. [WKS16]

1. Initialisation des Qubits

Chaque poids et chaque entrée du perceptron sont représentés par des qubits.

- Qubits de poids : $|w_1\rangle, |w_2\rangle, \dots, |w_n\rangle$
- Qubits d'entrée : $|x_1\rangle, |x_2\rangle, \dots, |x_n\rangle$

Les qubits de poids sont initialisés dans une superposition de tous les poids possibles en appliquant une porte Hadamard à chaque qubit :

$$|w_i\rangle \rightarrow \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

2. Superposition d'Entrées

Les entrées du perceptron sont également mises en superposition. Supposons que nous avons deux entrées x_1 et x_2 :

$$|x_1\rangle \rightarrow \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

$$|x_2\rangle \rightarrow \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

Cette étape permet au perceptron quantique de traiter simultanément toutes les configurations possibles des entrées.

3. Calcul de la Somme Pondérée

Le calcul de la somme pondérée z se fait en utilisant des portes quantiques pour effectuer des opérations linéaires sur les qubits d'entrée et de poids. L'opération est représentée par une série de portes quantiques qui réalisent la multiplication et l'addition nécessaires.

- Multiplication : Utiliser des portes contrôlées pour réaliser la multiplication des qubits de poids et d'entrée.

$$|x_i\rangle \otimes |w_i\rangle \rightarrow |x_i w_i\rangle$$

- Addition : Utiliser des portes d'addition quantique pour sommer les produits pondérés.

$$z = \sum_{i=1}^n w_i x_i$$

Le résultat de cette somme est stocké dans un qubit de sortie, par exemple $|z\rangle$.

4. Application de la Fonction d'Activation Quantique

La fonction d'activation classique, telle que la fonction de seuil, doit être adaptée pour le traitement quantique. Une fonction d'activation quantique peut être réalisée à l'aide de portes quantiques qui modifient l'état du qubit de sortie en fonction de la valeur de z .

- Fonction de Seuil Quantique : Une porte de seuil quantique (ou comparateur) peut être utilisée pour déterminer si z dépasse un certain seuil T :

$$|z\rangle \rightarrow |y\rangle$$

où $y = 1$ si $z \geq T$, sinon $y = 0$.

5. Optimisation des Poids avec l'Algorithme de Grover

Pour trouver les poids optimaux, on utilise l'algorithme de Grover pour rechercher dans l'espace des poids.

- Oracle Quantique : Un oracle quantique est une fonction qui marque les solutions correctes. Dans le contexte du perceptron, l'oracle marque les états de poids qui minimisent l'erreur de prédiction. Cet oracle peut être construit de manière à inverser la phase des états correspondant à une faible erreur.

$$\mathcal{O}|w\rangle|x\rangle|y\rangle = |w\rangle|x\rangle|(-1)^{f(w)}y\rangle$$

où $f(w) = 1$ pour les poids optimaux et 0 sinon.

- Amplification par Amplitude : L'algorithme de Grover est appliqué pour amplifier les amplitudes des états de qubits correspondant aux poids optimaux. Après $O(\sqrt{N})$ itérations (où N est le nombre total de configurations de poids possibles), les états optimaux sont amplifiés. [Gro96a]

6. Mesure et Extraction des Poids

Après avoir appliqué l'algorithme de Grover, les qubits de poids sont mesurés pour obtenir les valeurs optimales. La mesure effondre l'état superposé des qubits dans une configuration de poids spécifique, qui correspond aux poids optimisés pour le perceptron.

$$|w_{\text{optimal}}\rangle$$

Nous proposons deux approches quantiques de l'apprentissage du perceptron. La première approche se concentre sur l'aspect informatique du problème et la méthode proposée réduit quadratiquement l'échelle de la complexité de l'apprentissage par rapport au nombre de vecteurs d'apprentissage. Le second algorithme se concentre sur l'efficacité statistique, nous proposons un algorithme qui améliore quadratiquement l'échelle de l'algorithme d'apprentissage en ce qui concerne la marge entre les classes dans les données d'apprentissage. Ce dernier algorithme combine l'estimation quantique de l'amplitude dans l'interprétation de l'espace de version du problème d'apprentissage du perceptron. Nos approches mettent en évidence les compromis que l'on peut prendre en compte dans le développement d'algorithmes quantiques et les avantages ultimes de l'exécution de tâches d'apprentissage sur un ordinateur quantique.

Perceptron Quantique Online Le principe est le suivant :

- Trouver un point mal classé.
- Corriger les poids en conséquence.
- Répéter le processus jusqu'à ce qu'il n'y ait plus de points mal classés ou jusqu'à un nombre maximal d'itérations.

Voici l'algorithme comme trouvé dans la littérature [WKS16] :

Algorithm 1 Algorithme d'entraînement du Perceptron Quantique en mode "online"

```

Pour  $k = 1, \dots, \lceil \log_{3/4}(\gamma^2 * \epsilon) \rceil$  faire :
  Pour  $j = 1 : \lceil \log_c(1 / \sin(2 \sin^{-1}(1/\sqrt{N}))) \rceil$  faire :
    Tirer  $m$  uniformément dans  $\{0, \dots, \lceil c^2 \rceil\}$ .
    Préparer  $\Psi$ .
     $\Psi \leftarrow ((2\Psi\Psi^\dagger - I)F_n)^m \Psi$ .
    Mesurer  $\Psi$ , considérer que la valeur est  $u_q$ .
     $(\phi, y) \leftarrow U^c(q)$ .
    si  $f_w(\phi, y) = 1$  alors
      Retourne  $w' \leftarrow w + y\phi$ 
    Retourne  $w'$ .

```

Perceptron Quantique Space Le perceptron quantique Space utilise une approche géométrique basée sur des hyperplans aléatoires pour trouver une solution séparatrice.

Ainsi le principe devient :

- Dessiner des hyperplans aléatoires.
- Identifier un hyperplan qui sépare les données de manière satisfaisante.

L'algorithme complet [WKS16] :

Algorithm 2 Algorithme d'entraînement du Perceptron Quantique en mode "space"

```

Pour  $k = 1, \dots, \lceil \log_{3/4}(\epsilon) \rceil$  faire :
  Pour  $j = 1 : \lceil \log_c(1 / \sin(2 \sin^{-1}(1/\sqrt{K}))) \rceil$  faire :
    Tirer  $m$  uniformément dans  $\{0, \dots, \lceil c^2 \rceil\}$ .
    Préparer  $\Psi = \frac{1}{\sqrt{K}} \sum_{p=1}^K u_p \otimes W_0$ .
     $\Psi \leftarrow ((2\Psi\Psi^\dagger - I)C)^m \Psi$ .
    Mesurer  $\Psi$ , considérer que la valeur est  $u_q$ .
     $w \leftarrow V^c(q)$ .
    si  $f_w(\phi_\ell, y_\ell) = 0$  pour tout  $\ell \in \{1, \dots, N\}$  alors
      Retourne  $w$ 
    Retourne  $w = 0$ .

```

L'explication de ces derniers algorithmes n'est pas aisée et nous ne maîtrisons pas encore toutes les subtilités néanmoins nous allons tâcher d'apporter les outils nécessaires à leur compréhension :

Perceptron Online 1. Boucle externe sur ‘k’ :

Pour $k = 1, \dots, \lceil \log_{3/4}(\gamma^2 * \epsilon) \rceil$ **faire** :

- ‘k’ varie de 1 à un certain nombre dépendant de γ (un paramètre lié à la marge de classification) et ϵ (la précision souhaitée). Cette boucle contrôle le nombre de fois où l’algorithme va ajuster les poids.

2. Boucle interne sur ‘j’ :

Pour $j = 1 : \lceil \log_c(1 / \sin(2 \sin^{-1}(1/\sqrt{N}))) \rceil$ **faire** :

- ‘j’ varie de 1 à un certain nombre dépendant de ‘N’ (le nombre de qubits, ou la dimension du système quantique). Cette boucle contrôle le nombre de fois où une certaine opération quantique est répétée pour s’assurer que l’algorithme converge correctement.

3. Sélection aléatoire :

Tirer m uniformément dans $\{0, \dots, \lceil c^2 \rceil\}$.

- Un entier ‘m’ est tiré uniformément au hasard dans un intervalle allant de 0 à une valeur dépendant de ‘c’ (un paramètre de l’algorithme).

4. Préparation de l’état quantique :

Preparer Ψ . $\Psi \leftarrow ((2\Psi\Psi^\dagger - I)F_n)^m\Psi$.

- Un état quantique ψ est préparé. Cet état est ensuite modifié par une opération quantique (impliquant un réflecteur et une porte de Fourier quantique F_n) répétée ‘m’ fois. Cela correspond à l’opérateur de Grover.

5. Mesure de l’état quantique :

Mesurer Ψ , considérer que la valeur est u_q .

- L’état Ψ est mesuré, et le résultat de cette mesure est noté u_q .

6. Calcul des sorties :

$(\phi, y) \leftarrow U^c(q)$.

- Une fonction quantique ‘U’ est appliquée à ‘q’ (issu de la mesure) pour obtenir une paire (ϕ, y) .

7. Mise à jour des poids :

si $f_w(\phi, y) = 1$ **alors** **Retourne** $w' \leftarrow w + y\phi$

- Si une certaine condition est remplie (dépendant de ‘f_w’, la fonction du perceptron actuel appliquée à ϕ), les poids du perceptron ‘w’ sont mis à jour en ajoutant ‘y ϕ ’.

Et c’est similaire pour l’algorithme spacial.

3.4 Comparaisons

Deux datasets ont été utilisés : Iris et un second conçu spécialement pour perturber l’algorithme online en ajoutant une nouvelle dimension à chaque point.

	algo	N	gamma	train_size	score	nb_operations	dataset
0	Online perceptron	100	0.011011	0.1	0.981111	21.0	Iris
1	Version space perceptron	100	0.011011	0.1	0.967778	120.0	Iris
2	Version space quantum perceptron	100	0.011011	0.1	0.975556	128.0	Iris
3	Online quantum perceptron	100	0.011011	0.1	0.983333	50.0	Iris
4	Hybrid quantum perceptron	100	0.011011	0.1	0.956667	65.2	Iris

FIGURE 6 – Comparaison sur Iris

Ces premiers résultats nous portent à croire que les algorithmes quantiques et classiques sont au coude-à-coude en termes de performance. Les algorithmes spatiaux globalement les moins bons en termes de score/nb_opérations. Néanmoins, les algorithmes quantiques demandent beaucoup d'opérations, le online et l'hybride nécessitent plus de 2 fois plus de calculs que le perceptron online classique, qui a d'ailleurs les meilleures performances.

	algo	N	gamma	train_size	score	nb_operations	dataset
0	Online perceptron	200	0.070711	0.9	0.54	16471.0	Hard
1	Version space perceptron	200	0.070711	0.9	0.55	14580.0	Hard
2	Version space quantum perceptron	200	0.070711	0.9	0.54	8298.0	Hard
3	Online quantum perceptron	200	0.070711	0.9	0.485	1397.3	Hard
4	Hybrid quantum perceptron	200	0.070711	0.9	0.515	653.8	Hard

FIGURE 7 – Comparaison sur le second dataset

En analysant les performances sur le 2ème dataset, plus complexe, les résultats sont globalement mauvais : dans le cas où on choisirait tout au hasard le score serait de 0.50. Pour autant, à scores similaires les algorithmes quantiques ont ici nécessité au pire 2 fois moins de calculs, la tendance semble s'inverser.

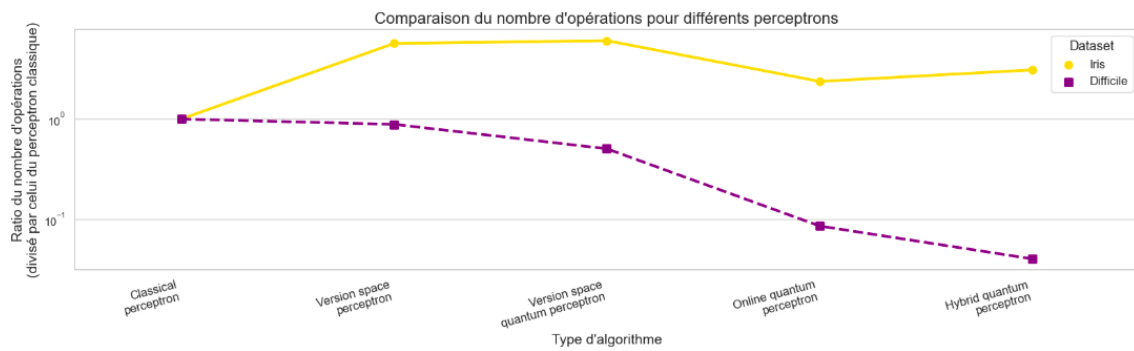


FIGURE 8 – Résultats globaux divisés par la performance d'un perceptron classique

Enfin, nous comparons les 2 tableaux précédents : l'ordonnée vaut 1 ou moins si le perceptron est aussi rapide ou meilleur que les algorithmes étudiés. Dans le cas du dataset Difficile, les performances laissent à désirer et les alternatives quantiques dominent bien, il semblerait que dans le cas où notre ordinateur quantique est capable de calculs précis et stables, ce serait un gain de temps formidable. Dans le cas d'Iris, c'est bonnet blanc et blanc bonnet, rien de notable.

Deuxième partie

Algorithmes quantiques sur ordinateur quantique

4 Quelques prérequis d'informatique quantique

Dans un premier temps, il convient d'expliciter les notions fondamentales nécessaires à la lecture de la suite du rapport. Bien que l'étude de ces notions eut été une partie incontournable de notre travail, cette section restera très concise. Nous aborderons donc brièvement le principe des qubits, les principales portes des ordinateurs quantiques, les hypothèses sur lesquels ces derniers se reposent, et enfin, l'état de Greenberger–Horne–Zeilinger.

4.1 Principe de base du qubit

Définition En principe, un qubit est un système quantique à deux niveaux, souvent notés $|+\rangle, |-\rangle, |\uparrow\rangle, |\downarrow\rangle$, ou encore $|1\rangle, |0\rangle$. C'est cette dernière notation qui est la plus commune en informatique quantique [Dur24].

Comme tout système quantique, ses états forment un espace de Hilbert représentable par des matrices, et auquel on peut associer un produit scalaire, une norme, un produit tensoriel, etc. Dans tous les cas dans ce rapport, nous utilisons les notations les plus canoniques.

Intérêt Tout l'intérêt des qubits par rapport aux bits classiques repose dans l'exploitation de leurs propriétés quantiques [JJ24]. Par exemple, un qubit peut avoir un état superposé avec deux composantes sur $|1\rangle$ et $|0\rangle$. De plus, si l'on se situe dans un système alliant plusieurs qubits, il est possible de faire en sorte que certains qubits soient intriqués entre eux, par exemple, en les faisant passer par une porte C-NOT¹.

Implémentations matérielles des qubits En vertu de la page publiée sur le sujet par IBM : [JJ24] Comme n'importe quel système quantique à deux niveaux peut être utilisé pour créer un qubit, il existe différents types de qubits actuellement développés par les chercheurs, et certains qubits sont mieux adaptés à certaines applications.

Citons donc les principaux types de qubits présentés par IBM :

- Les qubits supraconducteurs, fabriqués à partir de matériaux fonctionnant à des températures extrêmement basses et manipulés par des impulsions micro-ondes, sont appréciés pour leur cohérence robuste.
- Les qubits à ions piégés, utilisant une technologie laser sophistiquée, offrent des temps de cohérence prolongés et des mesures haute-fidélité.
- Les boîtes quantiques, semi-conducteurs capturant un seul électron, sont prometteuses pour leur évolutivité et compatibilité avec la technologie existante.
- Les qubits à photons, utilisant les états de spin des particules de lumière, permettent la communication quantique sur de longues distances.
- Les atomes neutres, excités par des lasers, fournissent des qubits adaptés à la mise à l'échelle et à certaines opérations quantiques.

4.2 Conditions nécessaires au fonctionnement d'un ordinateur quantique

Selon le cours de T. Durt [Dur24], DiVincenzo introduit cinq conditions nécessaires à la réussite des ordinateurs quantiques :

1. Un ordinateur doit pouvoir contenir un grand nombre de qubits (*scalable*).
2. Il doit être possible de préparer chaque qubit dans un état initial arbitraire.
3. Les portes quantiques universelles doivent y être implémentée afin de pouvoir réaliser toutes les opérations. Nous revenons sur ce point dans le paragraphe suivant (4.3).

1. Voir le paragraphe 4.3.3 à ce sujet.

4. Il faut travailler sur des temps plus courts que le temps de décohérence (afin de pouvoir faire interférer les particules entre-elles).
5. Il faut avoir la possibilité d'adresser les qubits séparément.

Si la plupart de ces conditions semblent évidentes, elles sont en réalité très difficiles à réaliser. Selon [Dur24], la décohérence est le plus gros problème que rencontreront les ordinateurs quantiques, ce problème est si important qu'il ne seront selon lui jamais opérationnels pour de réelles tâches. Enfin, on ajoute souvent deux autres conditions pour la communication utilisant des ordinateurs quantiques :

- La capacité d'interconvertir des qubits stationnaires et volants
- La capacité de transmettre fidèlement des qubits volants entre des emplacements spécifiés

Ces deux autres conditions ne sont en revanche **pas du tout** vérifiées dans les processeurs que l'on utilise dans notre étude, lesquels ne contiennent aucun qubit volant.

4.3 Principales portes des ordinateurs quantiques

Dans la suite, nous allons construire des circuits quantiques en utilisant des portes quantiques. Revenons donc ici sur les portes que nous allons utiliser, et introduisons rapidement chaque transformation quantique qui est subie par le système. Les informations sur les portes présentes dans ce paragraphe sont issues de [24b] et de [Dur24].

4.3.1 Portes de Pauli X, Y et Z

Parmi les portes à un qubit, les portes de Pauli X, Y et Z sont parmi les plus connues, notamment car leur réalisation matérielle est relativement simple selon [Dur24]. Ces trois portes ont la particularité de n'être que trois rotations selon trois axes différents.

Pauli-X La porte de Pauli X, aussi dite porte NOT quantique, est l'équivalent d'une porte NOT. Elle agit comme suit : $X|0\rangle = |1\rangle$ et $X|1\rangle = |0\rangle$. Sa représentation matricielle est donnée par $X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$.

Pauli-Y La porte de Pauli Y effectue une rotation autour de l'axe Y (de la sphère de Bloch) et est représentée par $Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$. Elle transforme $|0\rangle$ en $i|1\rangle$ et $|1\rangle$ en $-i|0\rangle$.

Pauli-Z La porte de Pauli Z correspond à une rotation autour de l'axe Z, avec $Z|0\rangle = |0\rangle$ et $Z|1\rangle = -|1\rangle$, et sa matrice est $Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$.

4.3.2 Porte de Hadamard

La porte de Hadamard (H) crée une superposition quantique en transformant les états de base de la manière suivante : $H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ et $H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$. Sa représentation matricielle est $H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$.

En pratique, nous l'avons souvent utilisée pour initialiser les qubits pour lesquels nous voulions que $|0\rangle$ et $|1\rangle$ soient équiprobables.

4.3.3 Porte C-NOT

Cette porte est quant-à-elle une porte à deux qubits. Elle agit en inversant le second qubit (cible) si le premier qubit (contrôle) est en état $|1\rangle$. La transformation CNOT est décrite par : $\text{CNOT}|00\rangle = |00\rangle$, $\text{CNOT}|01\rangle = |01\rangle$, $\text{CNOT}|10\rangle = |11\rangle$, et $\text{CNOT}|11\rangle = |10\rangle$. La matrice correspondante est

$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

Dans la pratique, cette porte est très utile pour générer des états intriqués. Elle est très largement utilisée en pratique puisque selon [24b], elles ont un relativement faible taux d'erreur. Rien

n'est certain puisque la documentation d'IBM indique le contraire. Une chose est cependant admise : tout dépend du processeur quantique utilisé. Comme on le verra au 5.2.2, c'est là qu'entrent en jeu les *transpilers*.

4.3.4 Autres portes

Porte de Toffoli On peut définir de manière similaire la porte de Toffoli (CCNOT) sur un système à 3 qubits où les deux premiers servent de contrôle, et qui est aussi représentée par une matrice de réflexion², cette fois-ci en dimension 8.

Porte contrôlée-U Ainsi de suite, on peut définir de manière générale la porte contrôlée-U, où $U = \begin{bmatrix} u_{00} & u_{01} \\ u_{10} & u_{11} \end{bmatrix}$ alors la porte contrôlée-U fonctionne sur deux qubits de telle sorte que le premier qubit sert de contrôle. La porte contrôlée-U relie les états basiques comme suit :

$$\begin{aligned} |00\rangle &\mapsto |00\rangle, & |01\rangle &\mapsto |01\rangle, \\ |10\rangle &\mapsto |1\rangle \otimes U|0\rangle = |1\rangle \otimes (u_{00}|0\rangle + u_{10}|1\rangle), & |11\rangle &\mapsto |1\rangle \otimes U|1\rangle = |1\rangle \otimes (u_{01}|0\rangle + u_{11}|1\rangle). \end{aligned}$$

La représentation matricielle de la porte contrôlée-U en dimension 4 est $C(U) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & u_{00} & u_{01} \\ 0 & 0 & u_{10} & u_{11} \end{bmatrix}$.

4.4 L'état Greenberger–Horne–Zeilinger (GHZ)

Les états Greenberger-Horne-Zeilinger (GHZ) sont un type d'états quantiques impliquant au moins trois qubits. L'état défini dans [24a] correspond par exemple à : $|\text{GHZ}\rangle = \frac{|000\rangle + |111\rangle}{\sqrt{2}}$.

Il s'agit d'un état intriqué, ce qui constitue son intérêt principal puisque l'on considère qu'il s'agit d'un état où les qubits sont "le plus" intriqués entre-eux et qu'il y a, toujours selon [24a], un relativement faible taux d'erreur lorsque l'on effectue cette opération.

5 Introduction au calcul quantique

5.1 Introduction aux différents modules de *qiskit*

Choix de qiskit Le choix de la librairie Qiskit, développée par IBM, présente plusieurs avantages par rapport à d'autres librairies de calcul quantique.

D'une part, Qiskit est une plateforme complète et open source, permettant la conception, la simulation et l'exécution de circuits quantiques sur des ordinateurs quantiques réels. D'autre part, avec plus de 550000 utilisateurs, cela en fait une des librairies les plus utilisées pour le calcul quantique.

Concurrence Google-IBM Enfin, concernant le calcul quantique, IBM et Google sont les principaux concurrents. Nous avons cependant remarqué, de notre expérience, que Google a un certain retard dans l'Open Source quantique.

En comparaison, IBM met régulièrement à jour ses processeurs quantiques et nous y donne un accès gratuitement, sans même préciser de contre-partie quant à l'utilisation que nous faisons de ces ordinateurs.

Qiskit permet d'ailleurs de faire appel à d'autres ordinateurs quantiques que ceux d'IBM, ce qui le rend d'autant plus ouvert encore.

5.1.1 Présentation des modules

Tout d'abord, présentons l'environnement dans lequel nous travaillons. En vertu de la page dédiée présentant l'API de qiskit [IBMd], nous allons revenir sur les différents modules présents dans l'écosystème qiskit que nous utilisons.

2. Plus précisément, il s'agit ici d'une matrice de réflexion en dimension 8 échangeant les coordonnées de la dimension 7 et de la dimension 8.

Les deux modules principaux issus de qiskit À partir des dernières versions de qiskit, deux modules principaux se démarquent de tous les autres, et permettent à eux seuls de réaliser la majorité des calculs que l'on souhaite.

- `pip install qiskit[visualization]`
- `pip install qiskit-ibm-runtime`

Ces deux modules correspondent en réalité au module principal de qiskit avec un environnement de visualisation (d'où proviennent d'ailleurs les schémas de nos circuits quantiques par la suite) ainsi que le module nous permettant de transpiler les circuits quantiques (et d'optimiser leur nombre d'opérations) en fonction des processeurs quantiques des machines d'IBM. `qiskit-ibm-runtime` permet aussi de s'authentifier et de mettre en file d'attente les codes avant qu'ils ne soient exécutés sur ordinateur quantique.

Autres modules utiles de qiskit Si les deux modules présentés précédemment sont les principaux utilisés, il est nécessaire de préciser que nous avons été amenés à en utiliser d'autres, tels que :

- `pip install qiskit-aer` permet la simulation de circuits quantiques sur des ordinateurs classiques, pour tester et développer des algorithmes quantiques.
- `pip install qiskit-machine-learning` intègre les techniques d'apprentissage automatique avec les circuits quantiques.
- `pip install qiskit-algorithms` propose une collection d'algorithmes quantiques pré-implémentés. Il semble que ce module commence à être vieux, mais il n'est pas déprécié.

Autres modules Enfin, nous avons naturellement été amenés à utiliser des modules python de base. Inutile de revenir plus en détails sur ceux-ci.

- `skikit-learn`
- `matplotlib.pyplot`
- `numpy`
- Autres modules classiques

5.1.2 Principes et exemple d'utilisation

Principes recommandés de programmation Qiskit et IBM recommandent très largement dans leur documentation les quelques principes suivants concernant la programmation de circuits quantiques :

1. Concevoir le problème avec un format nativement quantique. Construire le problème quantique.
2. Optimiser le circuit et les opérateurs
3. Exécuter en utilisant les fonctions primitives des processeurs quantiques.
4. Analyser les résultats.

Cadre de l'exemple³

Nous allons ici préparer un système à 3 qubits dans un état GHZ⁴. De là, nous allons d'abord effectuer 1000 mesures et tracer la distribution de quasi-pobabilité, puis, nous allons voir comment définir une observable et faire tourner un estimateur 1000 fois pour travers la valeur attendue.

Imports Tout d'abord, voici les imports effectués dans la suite du programme. Ces fonctions sont toutes issues du module *qiskit* dans sa version simple.

```
1 import numpy as np
2 from qiskit import QuantumCircuit
3 from qiskit.primitives.sampler import Sampler
4 from qiskit.quantum_info import SparsePauliOp
5 from qiskit.primitives import Estimator
```

3. Précisons, pour ces exemples et les suivants, que nos codes ont été indéniablement influencés par la documentation de qiskit.

4. Greenberger–Horne–Zeilinger, voir le paragraphe 4.4, qui en parle.

Préparation du système Tout d'abord, nous préparons notre système à 3 qubits dans l'état $|000\rangle + i|111\rangle$, lequel est un état de GHZ.

```
1 qc_example = QuantumCircuit(3) # un système quytantique de 3 qubits
2 qc_example.h(0) # Applique une porte de Hadamard au qubit 0
3 qc_example.p(np.pi / 2, 0) # ajout d'une phase (ici : rotation de pi/2 appliquée
  au qubit 0)
4 qc_example.cx(0, 1) # porte C-NOT entre le bit 0 et 1
5 qc_example.cx(0, 2)
```

nb : *QuantumCircuit* crée un nouveau circuit quantique, dans lequel les qubits sont adressés en fonction d'un numéro. C'est de là que sont issus les "0", "1", "2" que l'on retrouve dans les instructions suivantes. Notons par ailleurs que la rotation n'est ici que pour illustrer un exemple d'utilisation, on pourrait tout-à-fait faire un état de GHZ sans cette rotation, et c'est d'ailleurs ce que l'on fait dans l'exemple 5.2.4.

Mesure utilisant *Sampler* Dans un premier temps, voyons comment l'on effectue une mesure des différents états :

```
1 qc_measured = qc_example.measure_all(inplace=False) # inplace = False pour
  retourner un nouveau circuit
2
3 # Mesure du système
4 sampler = Sampler()
5 job = sampler.run(qc_measured, shots=1000) # effectue les opérations de mesure dé
  crites dans qc_measured
6 quasi_distribution = job.result().quasi_dists
```

La fonction *measure_all* ne mesure pas à proprement parler le système, elle prépare les configurations nécessaires à ce que le système soit mesuré. c'est en fait la fonction *Sample*, qui récupère chaque état possible (une mesure donc) du système. Ici on effectue 1000 tests.

On obtient un résultat proche de $\{0 : 0.5, 7 : 0.5\}$, où les états sont désignés par un numéro (ici 0 désigne $|000\rangle$ et 7 désigne $|111\rangle$). On note d'ailleurs que l'on mesure ici les "quasi-probabilités" des états, et non leur probabilité.

Observation utilisant *Estimator* Naturellement, effectuer une mesure d'amplitude n'est pas la seule chose qu'un ordinateur quantique devrait être capable de faire. Afin de pouvoir effectuer n'importe quelle observation, on peut créer une observable avec *SparsePauliOp* (basé sur Pauli-X, Pauli-Y et Pauli-Z, que l'on peut combiner et auxquels on peut ajouter des poids).

```
1 # Définition d'une observable
2 operator = SparsePauliOp.from_list([("XXY", 1), ("XYX", 1), ("YXX", 1), ("YYY", -1)
  ])
3
4 # Observation
5 estimator = Estimator()
6 job = estimator.run(qc_example, operator, shots=1000)
7 valeurs_attendues = job.result().values
```

Ici l'on effectue 1000 observations, et la valeur estimée que l'on obtient est $[4.0]$ (une liste ne contenant qu'un élément, qui est un float égal à 4).

5.2 Utilisation d'ordinateurs quantiques

5.2.1 Intérêt et mise en place de l'environnement

Sources et intérêt Bien que les simulations de circuits quantiques sur des ordinateurs classiques offrent un bon aperçu du comportement des algorithmes quantiques, elles sont limitées.

Dans toute cette aprtie, nous utilisons principalement la documentation d'IBM [IBMe] [IBMb] comme source, selon laquelle les limites des simulateurs d'ordinateurs quantiques sur ordinateurs classiques sont très facilement atteignables.

Confirmation par nos expériences De notre expérience, nous avons en effet observé beaucoup de différence entre les simulateurs d'ordinateurs quantiques et les ordinateurs quantiques.

Notamment, les simulations donnent des résultats beaucoup trop parfaits : les bruis sont gaussiens, il n'y a pas de recouvrement, et le résultat n'est pratiquement pas bruité.

D'autre part, les exécutions sur un réel ordinateur quantique ont des taux d'erreur assez élevés. On aborde plus en détail les caractéristiques des ordinateurs utilisés dans la partie 6.3.

Anecdote humoristique Il nous est même arrivé de réussir à faire planter un ordinateur quantique ! La chose s'est produite sur un système plutôt simple (il s'agissait du système à 100 qubits équirépartis dont on parle au paragraphe 5.2.4). Et comme souvent avec les ordinateurs quantiques, il a suffi de relancer le même *run* quelques minutes plus tard pour régler le problème.

Les ordinateurs quantiques sont donc fondamentalement différents de leurs simulateurs dans leurs comportements.

5.2.2 Transpilers

Intérêt Comme introduit au 4.3.3, les performances de chaque porte utilisée dépendent de l'architecture propre à chaque processeur quantique. C'est dans cette optique que les *Transpilers* s'illustrent.

Selon les portes qui sont implémentées dans un processeur, il peut être nécessaire de remplacer une rotation à 180° par deux rotations de 90° selon une direction, si certaines portes ne font pas présentes dans un processeur.

Aussi, il peut tout aussi bien s'avérer utile de passer un chemin par lequel l'on sait qu'il y aura un *error-bit-rate* inférieur. Par exemple pour `ibm_osaka`, (sur la figure 14) il peut être préférable de faire certaines transformations sur le qubit 14 plutôt que sur le qubit 16.

Ce que l'on vient d'évoquer correspond évidemment à une simplification grossière, mais l'on y comprend bien qu'en dehors de l'écriture d'un système quantique, l'implémentation pratique du circuit est différente. Cet effet est d'autant plus important à considérer lorsque l'on sait que traditionnellement, les *transpilers* changent parfois le langage dans lequel le code est programmé (comme le ferait un compilateur).

Fonctionnement des transpilers de qiskit Selon la documentation des transpilers d'IBM [IBMa], voici les six couches de fonctionnement d'un transpiler.

1. **init** - Cette étape exécute toutes les passes initiales nécessaires avant de commencer à intégrer le circuit au backend. Cela implique généralement le déroulement des instructions personnalisées et la conversion du circuit en portes à 1 et 2 qubits.
2. **layout** - Cette étape applique un agencement, en mappant les qubits virtuels dans le circuit aux qubits physiques sur un backend.
3. **routing** - Cette étape se déroule après l'application d'un agencement et injecte des portes supplémentaires (telles que des inversions) dans le circuit original pour le rendre compatible avec la connectivité du backend.
4. **translation** - Cette étape traduit les portes du circuit en l'ensemble de bases du backend cible.
5. **optimization** - Cette étape exécute une boucle principale d'optimisation de manière répétée jusqu'à ce qu'une condition (telle qu'une profondeur fixe) soit atteinte.
6. **scheduling** - Cette étape gère toutes les passes de planification tenant compte du matériel. Concrètement, elle ajoute du délai pour compenser certains temps d'exécutions de certaines portes qui ne sont pas tout-à-fait symétriques⁵.

Bien plus de détails sont disponibles dans la documentation correspondante (où chaque étape est détaillée).

5.2.3 Exemple minimal

Présentation Dans cet exemple, nous allons créer un circuit quantique avec deux qubits de façon similaire à ce que l'on a fait en 5.1.2. Par la suite, nous allons observer sa représentation schématique, choisir un ordinateur quantique, transcompiler pour cet ordinateur quantique, et observer le système quantique qui va réellement s'exécuter sur l'ordinateur.

Enfin, nous allons lancer l'exécution en utilisant des "primitives quantiques", récupérer les résultats et les analyser.

5. Une belle analogie est à voir ici avec les transistors présentés dans le cours sur le traitement matériel de l'information de Mme C. Fossati.

Initialisation D'une part, on initialise notre circuit comme expliqué plus tôt.

```

1 # Imports
2 from qiskit import QuantumCircuit
3 from qiskit.quantum_info import SparsePauliOp
4 from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager
5 from qiskit_ibm_runtime import EstimatorV2 as Estimator
6 from qiskit_ibm_runtime import QiskitRuntimeService
7 from qiskit.quantum_info import SparsePauliOp
8 from time import time
9
10 # Construction du circuit quantique
11 qc = QuantumCircuit(2)
12 qc.h(0)
13 qc.cx(0, 1)
14
15 # Initialisation des observables
16 observables_labels = ["IZ", "IX", "ZI", "XI", "ZZ", "XX"]
17 observables = [SparsePauliOp(label) for label in observables_labels] # Il y a au
    total 6 observables différentes.

```

Au passage, on initialise ici aussi les observables que l'on va utiliser par la suite. À l'aide de la méthode `qc.draw("mpl")`, on peut faire appel à une instance de matplotlib pour dessiner ce à quoi ressemble notre circuit. Voir la figure 9

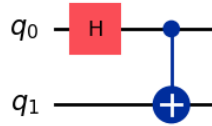


FIGURE 9 – État dans lequel est préparé notre circuit quantique

Transcompilation Désormais, il est temps de se connecter à notre compte IBM et de sélectionner un *Backend*, c'est-à-dire un ordinateur quantique. Par mesure de courtoisie, on choisit de manière générale le backend le moins chargé. Ici, le backend ayant été choisi au moment de l'exécution est `ibm_sherbrooke`.

```

1 # Connexion aux services d'IBM
2 QiskitRuntimeService.save_account(channel="ibm_quantum", token="<Token Confidentiel>"
    ", set_as_default=True, overwrite=True)
3
4 # Choix de l'ordinateur quantique
5 service = QiskitRuntimeService()
6 backend = service.least_busy(simulator=False, operational=True)
7
8 # Optimisation en fonction du backend choisi
9 pm = generate_preset_pass_manager(backend=backend, optimization_level=1)
10 isa_circuit = pm.run(qc)

```

Par la suite, l'optimisation est lancée. Nous avons choisi de mettre une `optimization_level = 1` pour obtenir un circuit relativement simple (même si, en soi, il est rare que la différence soit visible dans les diagrammes).

Global Phase: $7\pi/4$

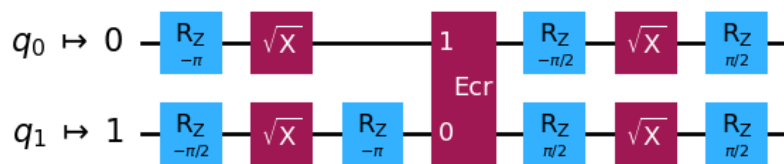


FIGURE 10 – Circuit optimisé pour `ibm_sherbrooke`

On peut observer à la figure 10 que spécifiquement pour `ibm_sherbrooke`, compte tenu des portes qui existent, il est préférable de faire plusieurs quarts de rotations sur chaque qubit plutôt que d'appliquer une porte de hadamard sur un seul d'entre eux. C'est logique puisque seule leur orientation relative compte pour la porte C-NOT qui suit. Enfin, quelques rotations sont effectuées avant la sortie afin de régulariser la situation par rapport aux éventuelles observables que nous pourrions vouloir utiliser.

Exécution En parlant d'observables, il est temps de les intégrer dans un estimateur afin de pouvoir lancer notre calcul quantique.

```
1 # Construction de l'estimateur
2 estimator = Estimator(backend=backend)
3 estimator.options.resilience_level = 1
4 estimator.options.default_shots = 5000
5
6 mapped_observables = [observable.apply_layout(isa_circuit.layout) for observable in
7                         observables]
8
9 # Exécution (mise en liste d'attente)
10 job = estimator.run([(isa_circuit, mapped_observables)])
```

Notons qu'ici `job` est un objet assez spécial. Il s'agit d'un objet possédant un `job_id`, qui permet de pouvoir récupérer une tâche après qu'elle ait fini d'être exécutée par les ordinateurs quantiques d'IBM. Par exemple, voici le `job_id` de la tâche que l'on vient d'exécuter : Job ID: `cs9jk44tg3bg008zthzg`. Ainsi, vous pouvez tout-à-fait récupérer les données de ce `job` dans le futur, et vous en servir plutôt que de réitérer un calcul déjà fait.

Résultats Enfin, il est nécessaire de pouvoir récupérer les résultats de notre `job`. La tâche en soi n'a rien de sorcier, mais il faut connaître la syntaxe :

```
1 # Récupération des résultats
2 job_result = job.result()
3 pub_result = job.result()[0]
4 values = pub_result.data.evs
5 errors = pub_result.data.stds
6
7 # Tracé des résultats
8 plt.plot(observables_labels, values, '-o')
9 plt.xlabel('Observables')
10 plt.ylabel('Values')
11 plt.show()
```

Par la suite, on peut effectuer un tracé (tout ce qu'il y a de plus conventionnel, avec `matplotlib`), ce qui nous donne la figure 11. On y observe par ailleurs que l'ordinateur de sherbrooke est particulièrement précis⁶

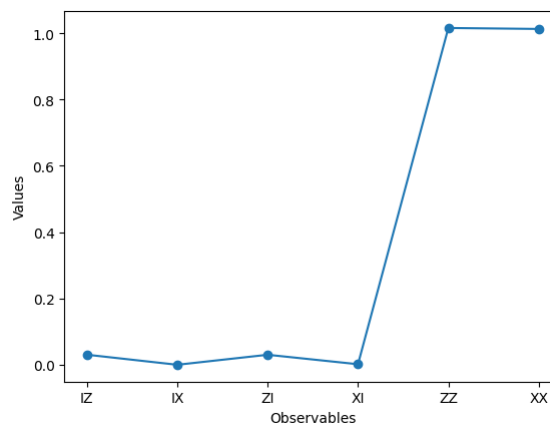


FIGURE 11 – Valeurs mesurées de chaque observable (quasi-probabilités)

6. Nous avons fait des tests préliminaires sur l'ordinateur d'osaka, et les composantes sur ZZ et XX étaient de l'ordre de 0.8, alors qu'ici on est légèrement au dessus de 1 (il s'agit de quasi-probabilités).

5.2.4 Exemple mettant en lumière le bruit des processeurs quantiques

Explication et initialisation Cet exemple est un approfondissement du précédent. Globalement on y fait la même chose, mais avec 100 qubits intriqués. Chaque qubit a est dans l'état de GHZ⁷ à 100 qubits pour un système à deux niveaux. On définit pour chaque position, une observable caractérisée par la distance entre le qubit 0 et le i-ième qubit, et on peut ensuite construire un estimateur, exécuter le programme et observer les résultats. La théorie nous prédit que l'on devrait avoir la même observation partout, indépendamment de la distance entre les qubits.

D'ailleurs, ce que l'on appelle "distance" ici correspond juste à l'indice du qubit i moins l'indice du qubit 0 : cela est juste une notation, et si notre ordinateur quantique fonctionnait bien, cela n'aurait absolument aucun impact.

Voici les imports utiles dans le reste de cet exemple :

```
1 from qiskit import QuantumCircuit
2 from qiskit.quantum_info import SparsePauliOp
3 from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager
4 from qiskit_ibm_runtime import QiskitRuntimeService, EstimatorOptions
5 from qiskit_ibm_runtime import EstimatorV2 as Estimator
6 import matplotlib.pyplot as plt
7 import numpy as np
```

Préparation du circuit Nous allons donc préparer nos 100 qubits dans un état de GHZ similaire à celui de l'exemple précédent. Plus précisément, le i-ième qubit est à l'état

$$|GHZ_i\rangle = \frac{1}{\sqrt{2}} (|0\dots 0\rangle + |1\dots 1\rangle) = \frac{1}{\sqrt{2}} (|0\rangle^{\otimes 100} + |1\rangle^{\otimes 100})$$

Concrètement pour implémenter cela en python, il suffit de faire une boucle. On passe le qubit 0 au travers d'une porte de Hadamard, et le reste des qubits subissent une C-NOT deux par deux.

```
1 n = 100
2 qc = QuantumCircuit(n)
3 qc.h(0)
4 for i in range(n-1):
5     qc.cx(i, i+1)
```

On peut tracer le résultat, ce qui donne la figure 12.

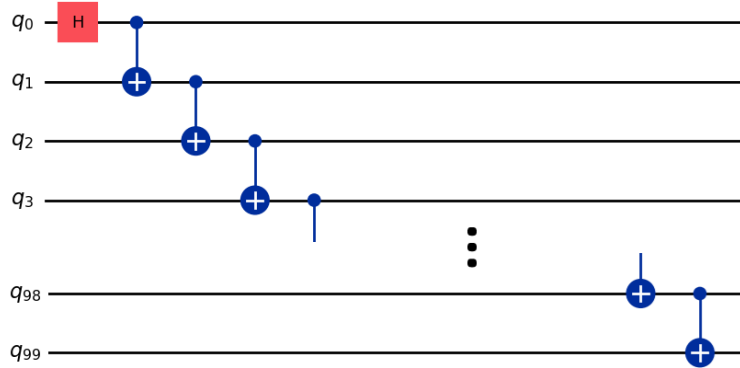


FIGURE 12 – Schéma (rogné) du circuit quantique

Préparation des observables Par la suite, nous allons définir 100 observables qui nous permettront de mesurer comment le qubit 0 influence chaque i-ième qubit. Plus précisément, on définit dans notre programme les 100 observables suivantes :

$$O_i = Z_0 \otimes \bigotimes_{j=1}^{99} (1 - \delta_{ij}) I_j + \delta_{ij} Z_j$$

7. Voir 4.4.

Nos notations semblent peut-être farfelues, mais conceptuellement, il n'y a pas de grande difficulté : la position de 0 est un Pauli-Z, et tous les autres sont des identités, sauf la i -ème dimension qui est aussi un Pauli-Z.

Voici donc l'implémentation en python de ces opérateurs :

```
1 operator_strings = ['Z' + 'I'*i + 'Z' + 'I'*(n-2-i) for i in range(n-1)]
2 operators = [SparsePauliOp(operator) for operator in operator_strings]
```

Optimisation matérielle On se connecte à IBM et on effectue ensuite notre optimisation comme précédemment.

```
1 service = QiskitRuntimeService()
2 backend = service.least_busy(simulator=False, operational=True, min_num_qubits=100)
3
4 pm = generate_preset_pass_manager(optimization_level=3, backend=backend) #
   environ 8 secondes pr optimization =3
5 isa_circuit = pm.run(qc)
6 isa_operators_list = [op.apply_layout(isa_circuit.layout) for op in operators]
```

Exécution On crée notre estimateur à partir des observables et de notre backend presque comme précédemment. On y ajoute en effet des options qui, après avoir épluché le code source correspondant, ne changent pratiquement rien, à part l'option *optimisation_level*, qui fait que l'estimateur ne s'adaptera pas aux hamiltonien des observables.

Ces options ont pour conséquence de simplifier un peu l'optimisation du circuit, si bien qu'on pourrait tracer sa représentation et comprendre ce qu'il fait. Nous n'allons évidemment pas faire cela car le plot est si grand qu'il dépasse les limites de taille de matplotlib.

```
1 options = EstimatorOptions()
2 options.resilience_level = 1
3 options.optimization_level = 0 # Ce 0 est en lien avec le résultat obtenu à la fin
4 options.dynamical_decoupling.enable = True
5 options.dynamical_decoupling.sequence_type = "XY4"
6
7 # Création de l'estimateur
8 estimator = Estimator(backend, options=options)
9
10 # Soumission du circuit
11 job = estimator.run([(isa_circuit, isa_operators_list)])
```

Résultats Désormais, le script suivant récupère les résultats, et calcule cette fameuse "distance". Il s'agit en fait uniquement de l'écart entre le numéro du qubit 0 et des autres qubits. Décrit comme cela, cette distance paraît absurde, et l'on pourrait se demander pourquoi nous la calculons, eh bien voici son tracé :

```
1 data = list(range(1, len(operators)+1)) # distance entre le qubit 0 et le qubit i (
   égale à i+1)
2 result = job.result()[0]
3 values = result.data.evs
4 values = [v / values[0] for v in values] # Normalisation des valeurs
```

Surprise : la valeur de l'observable dépend de cette distance qui étant pourtant purement abstraite. Cela illustre en réalité le bruit qu'il y a dans un ordinateur quantique. Si l'on vient à utiliser cette quantité de qubits, notre système est soumis à un bruit très important, si bien que même la deuxième observable a une amplitude relative en dessous de 0.7.

C'est très faible sachant que dans l'exemple 5.2.3, le seuil à partir duquel on considérerait l'expérience "ratée" était lorsque l'amplitude relative descendait en dessous de 0.8. Cela illustre combien dans un ordinateur quantique, le bruit augmente non seulement en fonction de la distance⁸, mais aussi en fonction de la quantité de qubits utilisée, ce qui est conforme à l'intuition, mais tout de même assez décourageant.

8. Cette fois-ci on parle certes de la distance des indices, mais l'on fait aussi référence à la distance matérielle entre les positions des qubits. Voir la figure 14 permet d'en avoir une représentation schématique.

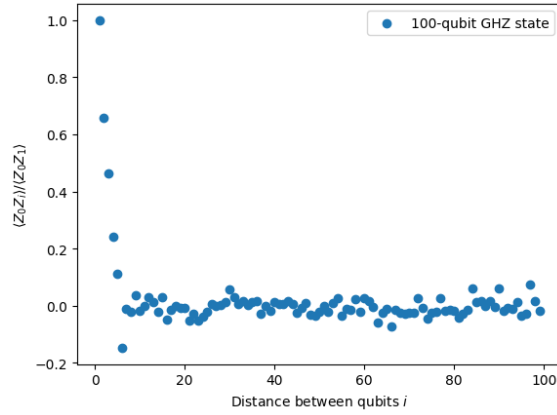


FIGURE 13 – Valeur de $Z_0 Z_i$ normalisée en fonction de la distance entre les qubits

6 Informations sur les ordinateurs quantiques utilisés

Nous allons donc utiliser par la suite plusieurs ordinateurs quantiques. Afin d'y voir plus clair sur leurs caractéristiques, nous allons ici décrire les ordinateurs utilisés, et définir les caractéristiques. Ceci inclut bien évidemment aussi les ordinateurs utilisés dans les exemple 5.2.3 et 5.2.4.

6.1 Principaux processeurs actuels d'IBM

Faisons rapidement la liste de processeurs utilisés par IBM dans leurs ordinateurs. Conformément à leur page dédiée aux processeurs, [IBMc], nous avons dressé un bref tableau des caractéristiques de chacun⁹.

Processeur	Qubits	Description
Canary	Jusqu'à 16	Réseau 2D optimisé
Falcon	Jusqu'à 27	Empilement multi-puce
Hummingbird	Jusqu'à 65	Améliorations de l'évolutivité ("Scalability")
Eagle (2021)	Jusqu'à 127	Évolutivité accrue
Osprey (2022)	Jusqu'à 433	Améliorations supplémentaires du packadge
Egret (2023)	Jusqu'à 33	Coupleurs réglables pour des portes plus rapides
Heron (2023)	Jusqu'à 133	Innovations dans la distribution des signaux
Condor	Jusqu'à 1,121	Nombre élevé de qubits (n'existe pas encore)

TABLE 1 – Résumé des caractéristiques des processeurs d'IBM

6.2 Error Per Layered Gate (EPLG)

L'EPLG est une mesure de la performance d'un processeur quantique utilisée par IBM. IBM décrit cette quantité comme une façon plus fiable d'évaluer les erreurs produites par un ordinateur quantique (par rapport à leur précédente mesure qui était le "Volume quantique") [IBMf].

S'il est relativement simple de se représenter cette mesure, il est en réalité très difficile de l'exprimer dans le cas général. Tout dépend du processeur, et les architectures changent très fréquemment. Fort-heureusement, il est possible de visualiser en temps réel les erreurs par porte dans chaque processeur d'IBM. Par exemple, regardez la figure 14.

Sur la figure 14, on observe une gradation par couleur de taux d'erreur par qubit. Par exemple, le qubit n°16 a un taux d'erreur à 4.93×10^{-1} et est donc très clair, tandis que le qubit n°57 a un taux d'erreur de 6.80×10^{-3} et est sombre. Enfin, notons que ce diagramme correspond à l'ordinateur d'Osaka, que nous avons choisi de représenter puisque c'est celui-ci que nous avons le plus souvent utilisé dans nos simulations.

9. Notons que nous avons ici tenté de traduire le plus fidèlement possible les mots qu'IBM utilise pour qualifier ses processeurs. Il advient alors que certaines phrases semblent soulever des arguments d'ordre commercial. N'hésitez pas à consulter [IBMc] pour obtenir les informations non-traduites.

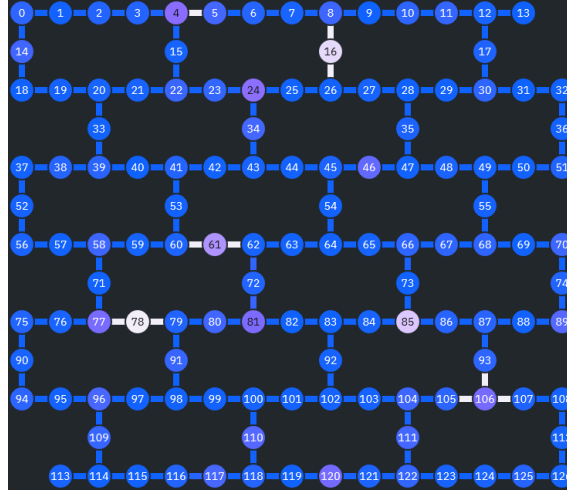


FIGURE 14 – Représentation des erreurs par qubit (*Readout assignment error*) d'`ibm_osaka`.

6.3 Caractéristiques des ordinateurs utilisés

Notons que dans le cadre de la partie II, nous utilisons les ordinateurs quantiques d'IBM ayant des processeurs *Eagle r3*, qui ont l'avantage d'avoir 127 qubits, mais ont malheureusement un EPLG¹⁰ assez insatisfaisant : allant de 1.9% à 3.6% pour les ordinateurs que nous utilisons. Relativisons tout de même : ces taux d'erreurs sont excellents, et le mieux qu'IBM est capable de fournir correspond à 0.8% d'EPLG pour son ordinateur `ibm_torino`, qui fonctionne avec le processeur *Heron r1*.

L'ordinateur quantique que nous avons le plus souvent utilisé correspond à l'`ibm_osaka`, dont on peut d'ailleurs voir le *readout assignment error* par qubit à la figure 14. Il a une ELPG de 2.8%.

Précisons enfin que, par courtoisie, nous utilisons la sélection *least busy* de *qiskit* afin de sélectionner l'ordinateur quantique le moins occupé lors de nos essais.

7 Exploration d'algorithmes quantiques

7.1 Algorithme de Grover

7.1.1 Principe

Intérêt et histoire L'algorithme de Grover a été développé par le physicien Lov Grover en 1996 [Gro96b]. Il s'agit d'un algorithme quantique de recherche d'un ou plusieurs éléments qui répondent à un critère donné parmi N éléments non classés en temps proportionnel à $\sqrt{N}$ et avec un espace de stockage proportionnel à $\log(N)$.

Il s'agit donc d'un algorithme beaucoup plus rapide que les algorithmes classiques, où il faut parcourir l'ensemble des éléments pour tester s'ils répondent au critère, impliquant un temps de calcul proportionnel à N . On gagne donc une amélioration quadratique du temps de calcul grâce à cet algorithme pour ordinateur quantique, ce qui constitue une avancée majeure.

L'algorithme de Grover, grâce à sa capacité à utiliser une vaste gamme de critères de recherche, possède un caractère universel. Cela en fait l'un des algorithmes les plus importants et certainement les plus utiles en informatique quantique.

Version moderne de l'algorithme

7.1.2 Implémentation du circuit quantique

Présentation [Wat23]

Dans les prochaines sous-parties, nous allons décrire le fonctionnement de l'algorithme de Grover et détailler en parallèle les différentes parties du code Python que nous avons réalisé pour

¹⁰. Voir le paragraphe 6.2 qui précède celui-ci.

Algorithm 3 Algorithme de Grover

```
Préparer un registre quantique de  $n$ -qubits dans l'état  $|0\rangle$  et un dans l'état  $|1\rangle$ 
Appliquez une porte de Hadamard à chaque qubit
Superposer les états pour obtenir un état  $|\psi\rangle$ 
 $|\psi_{Of}\rangle$  reçoit  $|\psi\rangle$ 
for Nombre d'itérations souhaitées do
     $|\psi_{Of}\rangle$  reçoit : Appliquer l'oracle associé à  $f$  à  $|\psi\rangle$ 
     $|\psi_{Of}\rangle$  reçoit : Symétrie de  $|\psi_{Of}\rangle$  par rapport à  $|\psi\rangle$ 
     $|\psi\rangle$  reçoit  $|\psi_{Of}\rangle$ 
end for
Mesurer  $|\psi\rangle$ 
```

implémenter cet algorithme. Ces parties sont très inspirées de la documentation d'IBM [Wat23] ainsi que du cours [Qua].

Initialisation On commence par préparer le $(n + 1)$ -qubit composé de : n états $|0\rangle$ et un état $|1\rangle$. On obtient ainsi un système dans l'état :

$$|\psi_0\rangle = |0\dots 0\rangle \cdot |1\rangle$$

On applique ensuite une porte de Hadamard à chacun de ces états. Après cette transformation, l'état du système quantique passe d'un état initial déterminé à une superposition uniforme de tous les états possibles, comme expliqué dans la section 4.3.2. Le système est alors dans l'état :

$$\begin{aligned} |\psi_1\rangle &= H^{\otimes n+1} |\psi_0\rangle \\ &= H^{\otimes n} |0\rangle \cdot H |1\rangle \\ &= \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} |k\rangle \cdot \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \\ &= \frac{1}{\sqrt{2^n}} (|0\rangle + |1\rangle + \dots + |k_0\rangle + \dots + |2^n - 1\rangle) \cdot \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \end{aligned}$$

où k_0 représente l'indice de l'état que l'on souhaite mesurer à la fin de l'algorithme.

Ainsi, chacun de ces états est mesurable de manière équiprobable. Pour pouvoir sélectionner l'état qui nous intéresse et in fine augmenter sa probabilité de détection par rapport aux autres états, on applique un oracle sur le système quantique.

Ceci est du texte

```
Texte _ $ % from qiskit.circuit.library import GroverOperator, MCMT, ZGate
```

Création de l'oracle Un oracle quantique est une boîte noire, un dispositif ou une fonction qui peut fournir des réponses à certaines questions spécifiques lorsqu'on lui présente une entrée. L'oracle peut être considéré comme une fonction booléenne qui prend un ensemble de qubits en entrée et retourne une valeur en fonction de ces entrées. En pratique, un oracle est souvent implémenté par un opérateur unitaire spécifique qui agit sur un registre de qubits. Pour un problème donné, cet opérateur unitaire est conçu pour répondre correctement aux questions posées par l'algorithme quantique. Les oracles jouent un rôle crucial en permettant aux algorithmes quantiques de manipuler et d'exploiter l'information de manière beaucoup plus efficace que ce qui est possible avec les méthodes classiques. En simulant des boîtes noires, ils permettent de concevoir des algorithmes qui peuvent résoudre des problèmes complexes avec un gain exponentiel en termes de vitesse ou d'efficacité par rapport aux approches classiques.

Dans le cadre de l'algorithme de Grover, l'oracle est une fonction qui permet de reconnaître la solution correcte en inversant la phase de l'état quantique correspondant à cette solution. Nous allons chercher à associer un oracle à la fonction f suivante :

Soit N un entier fixé et soit k_0 un entier avec $0 \leq k_0 \leq N - 1$. On définit la fonction

$$f : \{0, 1, \dots, N-1\} \rightarrow \{0, 1\}$$

$$k \mapsto \begin{cases} 1 & \text{si } k = k_0 \\ 0 & \text{si } k \neq k_0 \end{cases}$$

En effet, la solution mathématique du problème consiste à trouver l'antécédent de 1 par la fonction f . On aura alors l'indice k_0 de l'élément recherché.

L'oracle O_f réalise une fonction F définie comme suit :

Soit $x \in \mathbb{Z}/N\mathbb{Z}$ et $y \in \mathbb{Z}/2\mathbb{Z}$. Alors l'oracle réalise une fonction $F(x, y) = (x, y \oplus f(x))$, où $\oplus$ correspond à l'addition en $\mathbb{Z}/2\mathbb{Z}$ (qui vérifie $1 \oplus 1 = 0$). Si l'on écrit l'entier x à l'aide de son écriture binaire $x = x_1, x_2, \dots, x_n$, cela qui permet de récrire la fonction F sous la forme $F(x, y) = (x_1, \dots, x_n, y \oplus f(x_1, \dots, x_n))$.

L'application de l'oracle correspond alors au schéma quantique suivant :

Lorsque l'on applique l'oracle au qubit d'indice k , on obtient le résultat :

$$s = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \oplus f(k) = (-1)^{f(k)} \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = \begin{cases} \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) & \text{si } k \neq k_0 \\ -\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) & \text{si } k = k_0 \end{cases}$$

On a alors la transformation, à un facteur multiplicatif près :

$$|\psi_1\rangle = (|0\rangle + |1\rangle + \dots + |k_0\rangle + \dots + |2^n - 1\rangle) \rightarrow O_f|\psi_1\rangle = (|0\rangle + |1\rangle + \dots - |k_0\rangle + \dots + |2^n - 1\rangle)$$

et l'on peut distinguer à ce stade l'état que l'on recherche comme le montre la figure 15, où $|\phi\rangle$ représente l'état du système $|\psi_1\rangle$ sans l'état $|k_0\rangle$:

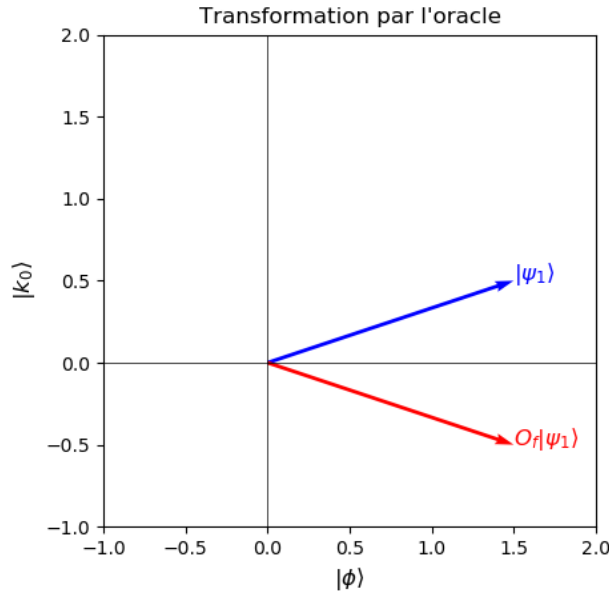


FIGURE 15 – Transformation réalisée par l'oracle

Il faut donc appliquer l'oracle pour chaque qubit afin de déterminer celui que l'on recherche. On voit donc bien l'intérêt de la physique quantique dans ce cas : on peut réaliser toutes ces opérations en parallèle, ce qui n'est pas le cas en physique classique car il aurait fallu tester chaque cas l'un après l'autre. C'est donc à cette étape que l'on voit l'avantage d'utiliser un ordinateur quantique par rapport à un classique.

Symétrie d'axe $|\psi_1\rangle$

La dernière étape fondamentale de l'algorithme de Grover est de réaliser la symétrie de l'état $O_f|\psi_1\rangle$ par rapport à l'axe $|\psi_1\rangle$.

Cette transformation peut s'écrire :

$$S_{\psi_1} = 2|\psi_1\rangle\langle\psi_1| - I$$

Ainsi, en appliquant cette transformation à $O_f|\psi_1\rangle$, que l'on note maintenant $|\psi_2\rangle$, on obtient :

$$S_{\psi_1}|\psi_2\rangle = 2|\psi_1\rangle\langle\psi_1||\psi_2\rangle - |\psi_2\rangle$$

Géométriquement, cette transformation correspond à la figure 16 :

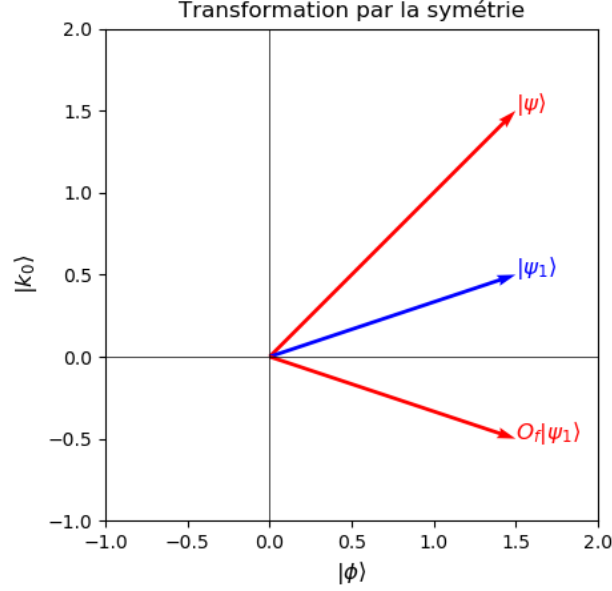


FIGURE 16 – Transformation réalisée par la symétrie

Plusieurs itérations de l'opérateur *GroverOperator* Ensuite, on réalise plusieurs itérations de la transformation de Grover (oracle et symétrie), de sorte à ce que l'amplitude de l'état $|k_0\rangle$ augmente à chaque itération. On peut le voir géométriquement sur la figure 17 :

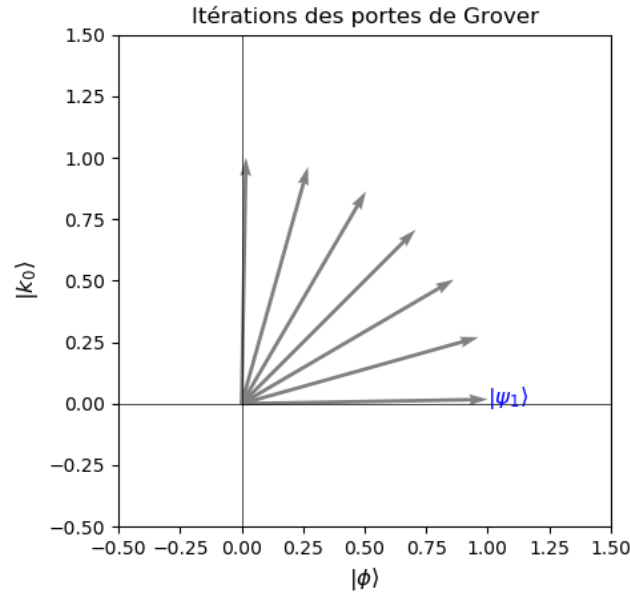


FIGURE 17 – Circuit de l'oracle obtenu

On observe qu'à chaque itération d'une porte de Grover, l'angle entre de $|\psi_n\rangle$ par rapport à $|\phi\rangle$ augmente, ce qui signifie que l'amplitude de l'état $|k_0\rangle$ augmente par rapport à celles des autres états. La mesure de $|k_0\rangle$ sera donc bien plus probable que celle des autres états à chaque nouvelle itération.

On montre que le nombre d'itérations optimal correspond à :

$$\left\lceil \frac{\pi}{4 \times \arcsin\left(\sqrt{\frac{n}{2q}}\right)} \right\rceil$$

Où n désigne le nombre d'états marqués par l'oracle : `len(marked_states)` et q désigne le nombre de qubits dans chaque opérateur de Grover : `grover_op.num_qubits`

L'amplitude de l'état $|k_0\rangle$ étant maintenant grande devant les amplitudes des autres états, on peut réaliser la mesure de $|\psi_{final}\rangle$ pour obtenir le n-bit k_0 avec une forte probabilité, et ainsi identifier le rang k_0 .

Le point faible fondamental de ce type d'algorithme est que la réponse qu'il fournit est probabiliste : le résultat renvoyé peut-être faux même si la probabilité que le résultat soit juste est très grande. De plus, la transformation de Grover génère du bruit sur les autres états en pratique, ce qui augmente légèrement les probabilités de mesurer le mauvais n-bit.

Implémentation de l'algorithme de Grover en Python Initialisation On importe les packages que l'on va utiliser de la même manière qu'en section 5.1.2

```
1 from qiskit import QuantumCircuit
2 from qiskit.circuit.library import GroverOperator, MCMT, ZGate
3 from qiskit.visualization import plot_distribution
4 from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager
5
6 from qiskit_ibm_runtime import QiskitRuntimeService
7 from qiskit_ibm_runtime import SamplerV2 as Sampler
```

On définit ensuite une fonction qui prend en argument une liste de qubits que l'on va manipuler. On commence par déclarer un nouveau circuit quantique en récupérant le nombre de qubits que l'on va utiliser.

```
1 def grover_oracle(marked_states):
2
3     # nombre de qubits dans le circuit
4     num_qubits = len(marked_states[0])
5     qc = QuantumCircuit(num_qubits)
6
7     # Marquage des états
8     for target in marked_states:
9         # inversion de la chaîne de caractères
10        rev_target = target[::-1]
11        # Liste des indices où l'état décrit un "0"
12        zero_inds = [ind for ind in range(num_qubits) if rev_target.startswith("0",
13        ind)]
14        # Ajout de portes Pauli-X sur les qubits à 0 avant et après l'ajout d'une
15        porte multi-contrôlée sur tout le monde
16        qc.x(zero_inds)
17        qc.compose(MCMT(ZGate(), num_qubits - 1, 1), inplace=True)
18        qc.x(zero_inds)
19    return qc
```

Dans la boucle `for`, on inverse d'abord les chaînes de caractères simplement car les bits de poids forts et de poids faibles sont intervertis dans la représentation donnée par `qiskit.visualization`.

On exécute ensuite dans l'ordre les instructions nécessaires à la réalisation de l'oracle quantique, données par `qiskit.visualization` dans la figure 18.

Un exemple d'utilisation de la fonction peut se faire avec la liste de chaînes de caractères `marked_states = ["011", "100"]`. On fait ensuite l'appel `oracle = grover_oracle(marked_states)`

On fait ensuite appel à la fonction

GroverOperator : `grover_op = GroverOperator(oracle)`.

Cette fonction est déjà implémentée dans le package `qiskit`. Elle permet d'appliquer un opérateur de Grover à l'état du système quantique que l'on a obtenu après l'appel de la fonction `grover_oracle`. Cet opérateur est décrit sur la figure 19. On va ensuite itérer l'appel à cette fonction un nombre optimal de fois.

Optimisation spécifique au backend Comme lors des précédents exemples, on utilise le 5.2.2 pour optimiser les opérations que l'ordinateur quantique devra réaliser en fonction de son architecture.

```
1 service = QiskitRuntimeService()
2 backend = service.least_busy(operational=True, simulator=False)
3
4 pm = generate_preset_pass_manager(target=backend.target, optimization_level=3)
5 circuit_isa = pm.run(qc)
```

Nos calculs ont ainsi été réalisés sur le backend `ibm_kyoto`.

Exécution Pour mesurer les états quantiques, on utilise une mesure de type 5.1.2 (Correspondant à l'exemple 5.1.2).

```
1 sampler = Sampler(backend=backend)
2 sampler.options.default_shots = 10_000
3 job = sampler.run([circuit_isa])
```

Résultats Il est enfin temps d'observer les résultats obtenus. Contrairement à l'exemple 5.2.3, nous utilisons ici une différente méthode de visualisation :

```
1 dist = job.result()[0].data.meas.get_counts()
2 plot_distribution(dist)
```

Il s'agit d'une méthode implémentée dans `qiskit.visualization`. Cela nous permet de tracer l'histogramme suivant :

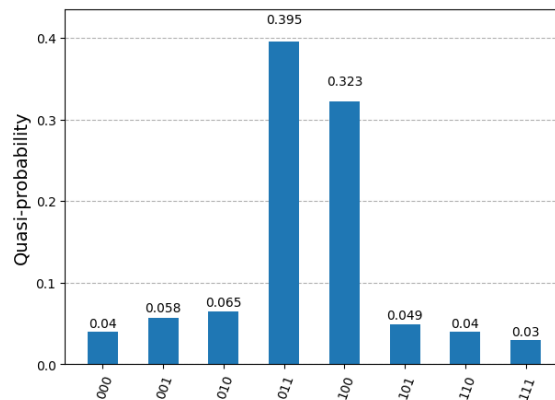


FIGURE 21 – Résultats des mesures après algorithme de Grover

On observe bien que les deux états marqués sont ceux mesurés avec les plus grandes quasi-probabilités, et que leurs amplitudes sont très grandes devant celles des autres états. On remarque également que la quasi-probabilité des autres états n'est pas nulle, ce qui montre que l'algorithme est bien probabiliste et que le résultat renvoyé peut être faux. L'implémentation de l'algorithme de Grover grâce au package `qiskit` a donc été réalisée avec succès.

7.2 Classification de type k-means

7.2.1 Problème du k-means quantique

Motivation Le k-means est un des algorithmes les plus classiques de *machine learning* non supervisé, et qui est relativement simple dans sa version basique. Dans nos sources de données (documentations, articles libres d'accès, etc), il ne nous a semblé trouver aucune implémentation entièrement quantique de ce programme.

En effet, tous les codes que nous avons pu lire à ce sujet utilisent d'une façon où d'une autre les bibliothèques de k-means classiques (par exemple `scikit-learn`). Nous n'avons en fait trouvé d'idées d'algorithme quantiques à réaliser sur machine quantique que dans certains articles.

Nous nous sommes donc lancés dans le projet de réussir à créer une version quantique sur ordinateur quantique de cet algorithme. Tout d’abord, il a fallu explorer plusieurs approches, telles que celles de [LMR13], qui nous a particulièrement influencée.

Principe proposé par Llyold et al. Passons l’initialisation du système. La méthode proposée par [LMR13] comprend grossièrement les phases suivantes :

1. Préparation des états quantiques : les points sont mis en mémoire de façon quantique¹¹.
2. Évaluation de distances par des tests de swap quantiques. Selon [LMR13], cela permet d’évaluer $\langle u | QFT | v \rangle$, et $\langle u | f(A) | v \rangle$ où A est une matrice Hermitienne, et QFT est une "transformée de Fourier quantique".
3. Assignation "adiabatique". Il est ensuite décrit un algorithme permettant d’assigner des points à un cluster. Cet algorithme est itératif et est supposé avoir un temps d’exécution total en $O(k \log(kMN))$ au maximum, et $O(\log(kMN))$ au minimum.

Faisabilité et adaptation Le lecteur attentif aura remarqué l’utilisation assez extensive de guillemets dans le précédent paragraphe. Les quelques lignes qui suivent relèvent de notre opinion personnelle.

D’une part, cet algorithme repose sur des hypothèses qui ne sont pas remplies aujourd’hui, telles que l’existence de mémoires RAM quantiques. Selon [Dur24], il n’est aujourd’hui possible de faire cela qu’avec quelques bits d’information, et ce n’est de toutes façons qu’en laboratoire et de façon instable.

En revanche, cet article est extrêmement enrichissant dans les idées qu’il propose. Notamment l’idée d’évaluation de la distance par opérations de swap quantiques.

Avancement du projet Nous avons donc cherché à effectuer des corrections à cet algorithme pour l’implémentation en pratique.

Nous n’allons pas revenir ici sur tous nos échecs, mais pour brièvement les résumer, nous avons exploré plusieurs pistes, et la seule qui a réussi à s’exécuter sans nous lancer une erreur, a rendu un résultat qui ne contenait qu’un seul cluster¹².

C’est donc un cuisant échec, et nous maquons cruellement de temps pour résoudre ce problème. Nous allons cependant exposer dans la partie qui suit, une bonne piste que nous avons (et qui fonctionne) pour calculer la distance euclidienne entre deux points de façon quantique.

7.2.2 Distance Euclidienne calculée par ordinateur quantique

Présentation du code Ce code calcule la distance euclidienne entre deux points d’un espace à 4 dimensions. Il a été inspiré de [Hel22] dans son concept et ses fondements théoriques, mais son implémentation *Qiskit* a dû être complètement revisitée, puisque le module a immensément évolué depuis 2022.

Concrètement, nous allons créer un système quantique composé de trois morceaux, qui intervertit la valeur de certains qubits pour retourner un angle.

Ensuite, nous faisons 10000 mesures, et l’on recalcule une distance à partir des quasi-probabilités obtenues.

Initialisation et environnement Tout d’abord, introduisons l’environnement dans lequel nous allons effectuer la suite du programme. Ici, on importe nos modules et on initialise notre instance comme dans les scripts précédents. Cette fois-ci, nos résultats seront issus d’`ibm_brisbane`.

```
1 import math as m
2 from qiskit import QuantumCircuit, ClassicalRegister, QuantumRegister
3 from qiskit.quantum_info import Statevector
4 from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager
5 from qiskit_ibm_runtime import QiskitRuntimeService
6 from qiskit_ibm_runtime import SamplerV2 as Sampler
7
8 service = QiskitRuntimeService()
9 backend = service.least_busy(operational=True, simulator=False)
```

11. Nous y revenons juste après, il s’agit là de la source du problème avec cet algorithme.

12. Alors qu’il y avait trois barycentres à placer. Précisons en plus que cet exemple avait été exécuté sur un simulateur, et que si l’on a obtenu ce résultat, c’est vraisemblablement car notre algorithme était mauvais.

```

10 target = backend.target
11 pm = generate_preset_pass_manager(target=target, optimization_level=3)

```

Quelques calculs déterministes Désormais, la suite du programme consiste en fonction prenant en arguments deux points en dimension 4.

Techniquement, nous avons codé la fonction pour une dimension n telle que $\exists k \text{ entier}, n = 2^k$. Cela se justifie par la suite par le fait que nous codons les angles de chaque dimension dans nos qubits.

Il nous faut donc débiter cette longue fonction par quelques calculs de base. Ces calculs nous permettent au passage de calculer la distance euclidienne de façon classique, et de pouvoir donc comparer les deux distances en sortie de notre fonction par la suite.

```

1 def calculate_distances(pointA, pointB, shots=10000):
2
3     n = len(pointA)
4     # Calcul des normes et des distances classiques
5     norm_A = sum([a**2 for a in pointA])
6     norm_B = sum([b**2 for b in pointB])
7     Dist = sum([(a - b)**2 for a, b in zip(pointA, pointB)])
8
9     Dist = m.sqrt(Dist)
10    A_norm = m.sqrt(norm_A)
11    B_norm = m.sqrt(norm_B)
12    Z = round(A_norm ** 2 + B_norm ** 2)
13
14    # Création des états phi et psi avec les données
15    phi = [A_norm / m.sqrt(Z), -B_norm / m.sqrt(Z)]
16    psi = []
17    for i in range(n):
18        psi.append((pointA[i] / A_norm) / m.sqrt(2))
19        psi.append((pointB[i] / B_norm) / m.sqrt(2))

```

Prenez note aussi de la variable Z , qui sera utilisée à la toute fin pour effectuer une opération de dilatation (sinon, on est restreints à avoir une distance environs¹³ comprise entre 0 et 1).

Création du circuit quantique Le moment est enfin venu de créer notre circuit quantique. Détaillons ce que l'on y fait.

```

1 # Structure du circuit quantique
2 q1 = QuantumRegister(1, name='q1')
3 q2 = QuantumRegister(n, name='q2')
4 c = ClassicalRegister(1, name='c') # pour faire notre mesure
5 qc = QuantumCircuit(q1, q2, c)
6
7 # Initialisation des états
8 qc.initialize(Statevector(phi), q2[0])
9 qc.initialize(Statevector(psi), q2[1:])
10
11 # Opérateur de test de swap
12 qc.h(q1[0])
13 qc.cswap(q1[0], q2[0], q2[1])
14 qc.h(q1[0])
15 qc.measure(q1, c)

```

Squelette du circuit Dans un premier temps, on crée la structure du circuit en trois morceaux :

1. un qubit de base, initialisé par une porte de Hadamard, qui sera le qubit mesuré en sortie.
2. un sous-circuit¹⁴ quantique composé ici de 4 qubits.
3. Une "couche" de mesure, qui est concrètement un bit classique.

Vous pouvez observer la figure 22 afin d'avoir une image plus claire du circuit final créé. On peut y observer que le sous-circuit est appelé "q2" et possède 4 qubits, par exemple.

13. "Environs" à cause de notre utilisation des quasi-probabilités.

14. Cette appellation est purement abstraite, elle permet de comprendre le principe du circuit. Il s'agit juste de qubits supplémentaires utilisés dans l'ordinateur, en pratique matériellement.

Notons que cette initialisation serait un point à améliorer dans une version future du programme. En effet, le manque de documentation disponible nous a amené à utiliser ces fonctions, mais qiskit étant un module qui évolue très vite, nous avons remarqué qu'il existait de nouvelles manières de faire ces opérations. Dans le futur, il serait approprié de rendre cette partie du programme la plus simple possible donc.

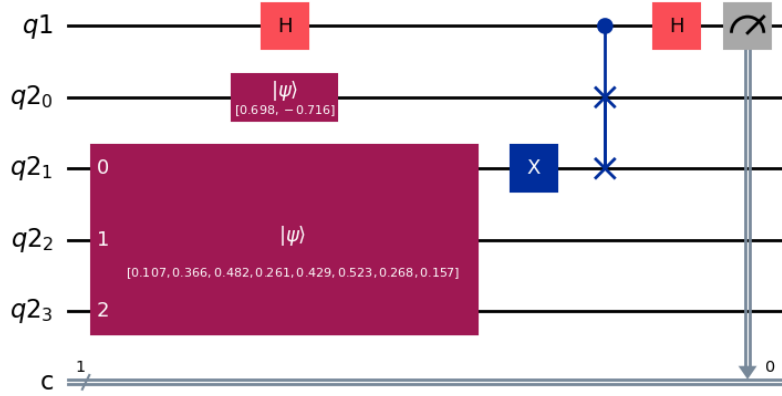


FIGURE 22 – Circuit calculant la distance entre deux points en dimension 4.

Initialisation des états et axe d'amélioration Ensuite, nous injectons dans le sous-circuit quantique les angles précédemment calculés.

Notons une petite subtilité : ces deux lignes de code injectent phi dans le premier qubit du sous-circuit, et injecte psi dans tous les autres qubits, sachant que psi est en fait une liste d'angles.

Il s'agit là d'un axe d'amélioration de ce programme : à l'heure actuelle, même s'il fonctionne pour toutes les dimensions de l'espace (pourvu qu'il s'agisse d'une puissance de 2), la dimension 2 produit une erreur à cause de cette ligne.

Nous n'avons cependant pas eu le temps de trouver une alternative propre, si bien que pour mesurer une distance en dimension 2, on se contentera de prendre des vecteurs dont la 3e et la 4e dimension ont les mêmes coordonnées (par exemple : 0).

Quantum Swap Nous arrivons ensuite à la partie du programme ayant le nom le plus vendeur. Le *Swap* n'est pourtant rien de plus qu'une inversion entre deux qubits.

Ici, il s'agit d'une version légèrement différente, puisque nous avons aussi un qubit de contrôle. Ici, le qubit de contrôle est notre qubit **initialisé en Hadamard** ! (il s'agit du qubit en première ligne sur la figure 22). On applique ensuite une nouvelle porte de Hadamard.

Même si jusque là, la raison du choix d'un cSwap était floue, on comprend immédiatement d'où provient la distance.

Mise en équation À titre indicatif, voici les quelques équations qui expliquent le phénomène que l'on va observer. Ps : les notations sont largement simplifiées.

Soient ϕ, ψ , deux angles. Nous allons nous placer dans un système à trois qubits (mesure de la distance en dimension 2) : le qubit de contrôle q_0 et les deux qubits de mesure $q_1 = |\psi\rangle, q_2 = |\phi\rangle$.

Après calculs, voici l'état de q_0 à la sortie du CSwap :

$$q_0 : \frac{1}{\sqrt{2}} (|0\psi\phi\rangle + |1\psi\phi\rangle)$$

Puis on y applique une porte de Hadamard :

$$q_0 : \frac{1}{2} (|0\psi\phi\rangle + |0\psi\phi\rangle + |1\psi\phi\rangle - |1\psi\phi\rangle)$$

Alors l'observation de q_0 donne :

$$P(q_0) = \left| \frac{1}{2} (|\psi\phi\rangle + |\phi\psi\rangle) \right|^2$$

$$= \frac{1}{4} (\langle\psi\phi|\psi\phi\rangle + \langle\psi\phi|\phi\psi\rangle + \langle\phi\psi|\psi\phi\rangle + \langle\phi\psi|\phi\psi\rangle)$$

Or, puisque $\langle\psi\phi|\phi\psi\rangle = \langle\psi|\phi\rangle \otimes \langle\phi|\psi\rangle = \langle\phi|\psi\rangle \otimes \langle\psi|\phi\rangle$, on obtient :

$$P(q_0) = \frac{1}{2} + \frac{1}{2} |\langle\psi|\phi\rangle|^2$$

Cela semble presque magique, mais la norme de la distance entre nos deux angles est apparue !

Mesures et résultats Il est enfin temps de récupérer nos résultats. Pour ceci, nous effectuons le calcul sur l'ordinateur quantique choisi dans l'initialisation, puis nous allons ajuster¹⁵ la valeur observée pour pouvoir obtenir la distance.

```

1  # Exécution du circuit quantique
2  sampler = Sampler(backend=backend) # modif faite pour circuit quantique réel
3  qc_transpiled = pm.run(qc)
4  job = sampler.run([qc_transpiled], shots=shots)
5
6  # résultats
7  job_result = job.result()
8  counts = job_result.quasi_dists[0]
9
10 # Calcul de la distance quantique
11 x = abs((job_result.quasi_dists[0].get('0', 0) - 0.5) * Z)
12 Q_Dist = round(m.sqrt(x), 4)

```

Si l'on exécute la fonction avec les points `pointA = [2, 9, 8, 5]` `pointB = [7, 5, 10, 3]`, on obtient le résultat suivant (exécuté sur `ibm_brisbane`) :

Distance quantique : 7.463 Distance Euclidienne : 7.0

Ceci conclut notre circuit de calcul d'une distance à l'aide d'un ordinateur quantique.

15. Le facteur $\frac{1}{2}$ provient de l'équation précédente, et l'on dilate ensuite en multipliant par Z. Évidemment, il faut ensuite passer le tout à la racine carrée.

Conclusion

Après avoir étudié les trois solutions possibles, il se trouve que le machine learning quantique sur un ordinateur classique est, dans certains cas, meilleur que les algorithmes classiques, surtout en termes de nombre d'opérations et notamment sur les jeux de données difficiles. Les algorithmes quantiques semblent donc meilleurs dans les cas qui les avantagent et sont légèrement moins performants dans les cas où ils sont désavantagés. Les algorithmes classiques sont plus robustes mais moins efficaces que les algorithmes quantiques.

De leur côté, les algorithmes quantiques exécutés sur des ordinateurs quantiques démontrent un potentiel significatif pour surpasser les méthodes classiques, en particulier pour des tâches spécifiques telles que l'optimisation et la factorisation. Cependant, ils sont aujourd'hui peu fonctionnels en raison de leur instabilité et de leur grand nombre d'erreurs.

Au cours de notre étude, nous avons été amenés à effectuer une certaine réflexion sur le futur des technologies que nous évoquons et que nous employons. cela nous a particulièrement inspiré dans nos recherches, et nous a apporté beaucoup. Comme l'on peut observer dans nos dernières parties, il est largement possible de poursuivre plus loin notre projet. Cependant, nous demeurons bien malheureusement limités par la capacité de calculs actuelle des ordinateurs quantiques, ainsi que par le fait que certaines technologies nécessaires sont encore inexistantes.

Ainsi, malgré un avenir prometteur, le machine learning quantique n'en est qu'à ses balbutiements. Il est nécessaire de poursuivre le développement des technologies quantiques pour exploiter pleinement leur potentiel et transformer les capacités de calcul futures.

Références

- [MP43] Warren S. MCCULLOCH et Walter PITTS. « A logical calculus of the ideas immanent in nervous activity ». en. In : *The Bulletin of Mathematical Biophysics* 5.4 (déc. 1943), p. 115-133. ISSN : 0007-4985, 1522-9602. DOI : [10.1007/BF02478259](https://doi.org/10.1007/BF02478259). URL : <http://link.springer.com/10.1007/BF02478259>.
- [Ros58] Frank ROSENBLATT. « The perceptron : a probabilistic model for information storage and organization in the brain. » In : *Psychological review* 65 6 (1958), p. 386-408. URL : <https://api.semanticscholar.org/CorpusID:12781225>.
- [Sol64] Ray J. SOLOMONOFF. « A Formal Theory of Inductive Inference. Part II ». In : *Inf. Control*. 7 (1964), p. 224-254. URL : <https://api.semanticscholar.org/CorpusID:1898103>.
- [CV95] Corinna CORTES et Vladimir VAPNIK. « Support-vector networks ». en. In : *Machine Learning* 20.3 (sept. 1995). ISSN : 0885-6125, 1573-0565. DOI : [10.1007/BF00994018](https://doi.org/10.1007/BF00994018). URL : <http://link.springer.com/10.1007/BF00994018>.
- [Gro96a] Lov K. GROVER. *A fast quantum mechanical algorithm for database search*. 1996. arXiv : [quant-ph/9605043](https://arxiv.org/abs/quant-ph/9605043) [[quant-ph](#)].
- [Gro96b] Lov K. GROVER. « A fast quantum mechanical algorithm for database search ». In : (1996). arXiv : [quant-ph/9605043](https://arxiv.org/abs/quant-ph/9605043) [[quant-ph](#)].
- [Bur98] Christopher J.C. BURGESS. « [No title found] ». In : *Data Mining and Knowledge Discovery* 2.2 (1998), p. 121-167. ISSN : 13845810. DOI : [10.1023/A:1009715923555](https://doi.org/10.1023/A:1009715923555). URL : <http://link.springer.com/10.1023/A:1009715923555>.
- [DHG01] Richard DUDA, Peter HART et David G.STORK. « Pattern Classification ». In : t. xx. Jan. 2001. ISBN : 0-471-05669-3.
- [LMR13] Seth LLOYD, Masoud MOHSENI et Patrick REBENTROST. *Quantum algorithms for supervised and unsupervised machine learning*. arXiv :1307.0411 [[quant-ph](#)]. Nov. 2013. URL : <http://arxiv.org/abs/1307.0411> (visité le 05/2024).
- [SSP14] Maria SCHULD, Ilya SINAYSKIY et Francesco PETRUCCIONE. « An introduction to quantum machine learning ». In : *Contemporary Physics* 56.2 (oct. 2014). ISSN : 1366-5812. DOI : [10.1080/00107514.2014.964942](https://doi.org/10.1080/00107514.2014.964942). URL : <http://dx.doi.org/10.1080/00107514.2014.964942>.
- [WKS16] Nathan WIEBE, Ashish KAPOOR et Krysta SVORE. « Quantum Perceptron Models ». In : (fév. 2016).
- [Kan+18] Thirunavukkarasu KANNAPIRAN et al. « Classification of IRIS Dataset using Classification Based KNN Algorithm in Supervised Learning ». In : déc. 2018, p. 1-4. DOI : [10.1109/CCAA.2018.8777643](https://doi.org/10.1109/CCAA.2018.8777643).
- [Par+20] Jae-Eun PARK et al. *Practical application improvement to Quantum SVM : theory to practice*. 2020. arXiv : [2012.07725](https://arxiv.org/abs/2012.07725) [[quant-ph](#)].
- [Hel22] HELEN URGELLES. *Calculate Quantum Euclidean Distance with Qiskit*. en. Oct. 2022. URL : <https://medium.com/qiskit/calculate-quantum-euclidean-distance-with-qiskit-df85525ab485> (visité le 27/05/2024).
- [Sha+22] Zheng SHAN et al. *Demonstration of Breast Cancer Detection Using QSVM on IBM Quantum Processors*. Mars 2022. DOI : [10.21203/rs.3.rs-1434074/v1](https://doi.org/10.21203/rs.3.rs-1434074/v1).
- [Sim22] Osvaldo SIMEONE. *An Introduction to Quantum Machine Learning for Engineers*. 2022. arXiv : [2205.09510](https://arxiv.org/abs/2205.09510) [[quant-ph](#)].
- [OK23] Zeynep OZPOLAT et Murat KARABATAK. « Exploring the Potential of Quantum-Based Machine Learning : A Comparative Study of QSVM and Classical Machine Learning Algorithms ». In : mai 2023, p. 1-5. DOI : [10.1109/ISDFS58141.2023.10131821](https://doi.org/10.1109/ISDFS58141.2023.10131821).
- [Wat23] John WATROUS. *Grover's algorithm / IBM Quantum Learning*. Juill. 2023. URL : <https://learning.quantum.ibm.com/tutorial/grovers-algorithm> (visité le 05/2024).
- [Dur24] Thomas DURT. *Cours d'information quantique, SISN*. fr-fr. Mai 2024.
- [24a] *Greenberger–Horne–Zeilinger state*. en. Page Version ID : 1215904698. Mars 2024. URL : https://en.wikipedia.org/w/index.php?title=Greenberger%E2%80%93Horne%E2%80%93Zeilinger_state&oldid=1215904698 (visité le 05/2024).

- [JI24] JOSH SCHNEIDER et IAN SMALLEY. *Qu'est-ce qu'un qubit ? / IBM*. fr-fr. Fév. 2024. URL : <https://www.ibm.com/fr-fr/topics/qubit> (visité le 01/05/2024).
- [24b] *Porte quantique*. fr. Page Version ID : 212803070. Fév. 2024. URL : https://fr.wikipedia.org/w/index.php?title=Porte_quantique&oldid=212803070 (visité le 05/2024).
- [SHM24] Teppei SUZUKI, Takashi HASEBE et Tsubasa MIYAZAKI. *Quantum support vector machines for classification and regression on a trapped-ion quantum computer*. 2024. arXiv : [2307.02091](https://arxiv.org/abs/2307.02091) [quant-ph].
- [IBMa] IBM. *API reference for qiskit.transpiler*. en. URL : <https://docs.quantum.ibm.com/api/qiskit/transpiler> (visité le 26/05/2024).
- [IBMb] IBM. *Instances*. en. URL : <https://docs.quantum.ibm.com/run/instances#open-plan> (visité le 05/2024).
- [IBMc] IBM. *Processor types*. en. URL : <https://docs.quantum.ibm.com/run/processor-types> (visité le 01/05/2024).
- [IBMd] IBM. *qiskit*. en. URL : <https://docs.quantum.ibm.com/api/qiskit> (visité le 05/2024).
- [IBMe] IBM. *Submit transpiled circuits / IBM Quantum Learning*. URL : <https://learning.quantum.ibm.com/tutorial/submit-transpiled-circuits> (visité le 05/2024).
- [IBMf] IBM. *Updating how we measure quantum quality and speed / IBM Quantum Computing Blog*. en. URL : <https://ibm.com/quantum/blog/quantum-metric-layer-fidelity> (visité le 26/05/2024).
- [Qua] QUANTUM. *Algorithme de Grover*. URL : <https://exo7math.github.io/quantum-exo7/grover/grover.pdf>.